NGUYEN DEVELOPMENT · PHPDBADMIN

Evolutionary Development — phpdbadmin

Timeboxes und Architekturentscheide ADR-001 bis ADR-011

Phase	04 Evolutionary Development
Träger	Nguyen Development (Einzelfirma), Hoang Nguyen · Lizenz MIT
Vorgehen	DSDM/AgilePM — Dokumentation in dokumentationen/, ein PDF je Phase
Stand	29.10.2026, 02:01 · baut auf Commit d6f6d40 + Änderungen dieses Standes auf
Quelle	dokumentationen/ im Repo — dieses PDF wird bei jeder Änderung neu gebaut

Dieses Dossier enthält

1. 04 — Evolutionary Development
2. Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen
3. Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber
4. Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber
5. Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf
6. Anhang E — ADR-005 — SQL-Editor: CodeMirror 6
7. Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel
8. Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund
9. Anhang H — ADR-008 — Anmeldung ohne Passwort
10. Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe
11. Anhang J — ADR-010 — Lizenz MIT
12. Anhang K — ADR-011 — PHP 8.5 als Mindestversion
13. Anhang L — Timebox 00 — Fundament
14. Anhang M — Timebox 01 — Katalog und Navigator
15. Anhang N — Timebox 02 — Datenraster und Filter
16. Anhang O — Timebox 03 — SQL-Editor und Befehlspalette → Alpha
17. Anhang P — Timebox 05 — Struktur ändern
18. Anhang Q — Timebox 09 — Server-Verwaltung

Inhalt

04 — Evolutionary Development
Timeboxes
Architekturentscheide

Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen
Kontext
Entscheid
Optionen
Konsequenzen

Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber
Kontext
Entscheid
Optionen
Konsequenzen

Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber
Kontext
Entscheid
Optionen
Konsequenzen

Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf
Kontext
Entscheid
Optionen
Konsequenzen

Anhang E — ADR-005 — SQL-Editor: CodeMirror 6
Kontext
Entscheid
Optionen
Konsequenzen

Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel
Kontext
Entscheid
Optionen
Konsequenzen

Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund
Kontext
Entscheid
Optionen
Konsequenzen

Anhang H — ADR-008 — Anmeldung ohne Passwort
Kontext
Entscheid
Optionen
Konsequenzen
Nachtrag TB 00 (28.10.2026)

Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe
Kontext
Entscheid
Optionen
Konsequenzen

Anhang J — ADR-010 — Lizenz MIT
Kontext
Entscheid
Optionen
Konsequenzen

Anhang K — ADR-011 — PHP 8.5 als Mindestversion
Kontext
Entscheid
Optionen
Konsequenzen

Anhang L — Timebox 00 — Fundament

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Browser-Durchlauf
Entscheide in der Timebox
Commits
Offen
Gelernt

Anhang M — Timebox 01 — Katalog und Navigator

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Commits
Gelernt

Anhang N — Timebox 02 — Datenraster und Filter

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Commits
Gelernt

Anhang O — Timebox 03 — SQL-Editor und Befehlspalette → Alpha

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Commits
Gelernt

Anhang P — Timebox 05 — Struktur ändern

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Entscheide in der Timebox
Tests
Commits
Offen
Gelernt

Anhang Q — Timebox 09 — Server-Verwaltung

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Gestaltungsentscheide (statt Wireframe-Abweichungen)
Entscheide in der Timebox
Tests
Commits
Offen
Gelernt

04 — Evolutionary Development

> **PDF der Phase:** evolutionary-development.pdf — bei jeder Änderung neu gebaut (`python3 dokumentationen/werkzeuge/pdf-bauen.py --nur 04`).

Timebox-Protokolle (`timebox-nn-titel.md`) und Architekturentscheide (`adr/ADR-nnn-titel.md`: Kontext → Optionen → Entscheid → Konsequenzen).

Timeboxes

TB	Inhalt	Anforderungen	Stand
00 Fundament	Starter-Kit bereinigt (kein Passwort, kein Dashboard, kein Profil); Anmeldung mit Code per Mail und Passkey; Einladung; Ersteinrichtung per Konsole mit Link und Passkey-Onboarding; App-Datenbank; Verbindungen mit Test, Verschlüsselung und erlaubten Hosts; Dialekt-Schicht mit Vertragstest auf vier Datenbanken; lokales Prüfskript, Workflow sparsam	A-01 bis A-06, A-09, V-01 bis V-03, V-05, B-01 bis B-03, B-06, B-08	✅ 28.10.2026 — offen: Touch ID von Hand (A-03), erster Lauf des Workflows auf GitHub
01 Katalog und Navigator	Arbeitsbildschirm mit Navigator für vier Datenbanken, «alle Datenbanken», Strukturansicht mit Art je Spalte, Schreibschutz und Umgebung, Kontowarnung, Deutsch und Englisch	O-01, O-04, S-01, V-04, N-06, V-10, O-06, N-04	✅ 29.10.2026 — V-06 mit TB 07; N-04-Matrix noch nie gelaufen
02 Datenraster und Filter	Raster mit Blättern über den Schlüssel, NULL zuletzt, Filtern, Suche, Zählen, Fremdschlüssel in beide Richtungen, Zeilen-Detail, Reiter, Kürzel; erste Seite bei 1 Mio. Zeilen unter 20 ms	D-01, D-02, D-03, D-06, N-01, O-03, O-05, O-09	✅ 29.10.2026 — «?»-Taste von Hand offen
03 SQL-Editor und Befehlspalette	Zerleger je Dialekt, Ausführen mit Zeit- und Zeilengrenze, Abbrechen auf dem Server, SQLite im Arbeitsprozess, Editor mit Vervollständigung, Verlauf, gespeicherte Abfragen, gefährliche Anweisungen nur mit Bestätigung, Befehlspalette ⌘K	Q-01–Q-04, Q-07, Q-09, O-02	✅ 29.10.2026 → **Alpha** — Woche im Alltag offen (Hoang)
05 Struktur ändern	Tabelle anlegen und ändern mit Vorschau aus `toSql()`, angewendet je Treiber (MySQL/MariaDB Anweisung für Anweisung, PostgreSQL in einer Transaktion, SQLite nach dem Umbauverfahren mit Checks, Triggern und Sichten), umbenennen, kopieren, leeren und löschen nur mit Namen, Datenbank bzw. Schema anlegen und löschen, neue SQLite-Datei; Tests je Treiber mit Elterntabelle und Kindern und mit Fehler mitten in der Folge	S-02–S-05, N-03	✅ 29.10.2026 auf eigenem Zweig, parallel zu TB 04 — offen: Browser-Durchlauf
09 Server-Verwaltung	Übersicht, Prozesse beenden, Variablen, Konten und Rechte mit Vorschau, Wartung — vier Treiber, Dialekt-Vertrag erweitert, jede Änderung genau ein Eintrag im Protokoll	W-01 bis W-05	✅ 29.10.2026 auf eigenem Zweig, vorgezogen — offen: Browser-Durchlauf

Architekturentscheide

Ausformuliert in der Foundation am 28.10.2026 (Kontext → Optionen → Entscheid → Konsequenzen), Grundlage Feasibility und Solution-Architektur:

ADR	Entscheid	Status
ADR-001	Dialekt-Schicht: ein Vertrag, vier Umsetzungen	angenommen
ADR-002	Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber	angenommen
ADR-003	Vorschau aus `toSql()`, Anwenden je Treiber	angenommen
ADR-004	Schreibschutz in Schichten, eine Anweisung je Aufruf	angenommen
ADR-005	SQL-Editor CodeMirror 6	angenommen
ADR-006	Raster TanStack Virtual, Blättern über den Schlüssel	angenommen
ADR-007	Export und Import in Paketen im Hintergrund	angenommen
ADR-008	Anmeldung ohne Passwort	angenommen
ADR-009	App-Datenbank SQLite als Vorgabe	angenommen
ADR-010	Lizenz MIT	angenommen
ADR-011	PHP 8.5 als Mindestversion	angenommen (Hoang 28.10.2026)

Übernahme aus der Feasibility (erledigt in TB 00): Die Proben F-1, F-7 und Teile von F-2 sind Vertragstests der Dialekt-Schicht (`tests/Feature/Dialects`); die Probe-Seiten `routes/probe.php`, `resources/js/pages/probe/*` bleiben lokal hinter der Anmeldung, bis Raster und SQL-Editor sie ersetzen — die Raster-Probe ist seit TB 02 entfernt, die Editor-Probe folgt mit TB 03.

Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-1, F-3)
Betrifft	Katalog, Struktur, SQL-Editor, Server-Verwaltung

Kontext

phpdbadmin soll vier Datenbanken mit derselben Bedienung abdecken (P-01). Laravel 13 liefert Verbindungen, Query Builder, Schema-Grammatiken und Introspektion für alle vier; die Feasibility zeigt, wo es nicht reicht: Check-Constraints, Trigger, Routinen, eigene Typen (`getTypes()` nur PostgreSQL), Prozesse, Rechte, Zeitlimits, Kontorechte. Ausserdem liefert MariaDB ohne Schemaangabe die Tabellen aller Datenbanken des Servers (3'603 statt 1'000).

Optionen

Option	Vorteile	Nachteile
A — eigene Abstraktion für alles	volle Kontrolle	baut nach, was Laravel schon kann und testet; viermal so viel Code
B — Laravel zuerst, Dialekt nur für Unterschiede	wenig eigener Code, Laravel-Updates kommen mit	Grenze zwischen Laravel und Dialekt muss klar sein
C — Paket eines Dritten (z. B. Adminer-Treiber)	viel fertig	fremde Struktur, Lizenzfrage, Bedienkonzept passt nicht

Entscheid

Option B. Ein Vertrag `App\Dialects\Dialect` mit vier Umsetzungen (`MySqlDialect`, `MariaDbDialect` als Ableitung, `PostgresDialect`, `SqliteDialect`). Was Laravel kann, ruft der Dienst direkt; der Dialekt liefert nur, was sich unterscheidet. Der Katalog übergibt **immer** das Schema.

Konsequenzen

- Jede Dialekt-Methode hat einen Vertragstest, der gegen alle vier Herd-Dienste läuft; die Proben aus `tests/Feature/Feasibility` werden dazu umgebaut.
- Neue Datenbanken (SQL Server, R-14) bekommen eine fünfte Umsetzung, die Oberfläche bleibt.
- Typnamen erscheinen im Original und mit einem gemeinsamen Oberbegriff.

Siehe Solution-Architektur · Feasibility

Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-2, F-8)
Betrifft	ConnectionFactory, QueryRunner

Kontext

Zugangsdaten liegen verschlüsselt in der App-Datenbank. Jede Anfrage braucht eine Verbindung mit Zeitzone UTC, Zeitlimit und gegebenenfalls Schreibschutz. Die Feasibility zeigt grosse Unterschiede: MySQL begrenzt nur SELECT und bricht still ab (Ergebnis `1`), MariaDB und PostgreSQL melden 70100 bzw. 57014, SQLite kennt kein Zeitlimit; libpq hält ganze Ergebnisse ausserhalb des PHP-Speichers (+105 MB bei einer Million Zeilen).

Optionen

Option	Vorteile	Nachteile
A — Verbindungen in `config/database.php` eintragen	einfach	Neustart je Verbindung, Geheimnisse in Dateien
B — `DB::build()` je Anfrage aus der App-Datenbank	keine Dateien, Name je Verbindung und Konto, `DB::purge` danach	Verbindungsaufbau je Anfrage (lokal ~1 ms)
C — dauerhafte Verbindungen (Pool)	schneller	PHP-FPM hat keinen gemeinsamen Pool; Zustand zwischen Konten

Entscheid

Option B. `ConnectionFactory` baut je Anfrage mit `timezone`, Zeitlimit (`max_execution_time`, `max_statement_time`, `statement_timeout`) und Optionen. Zeilengrenze je Treiber: MySQL/MariaDB ungepuffert + `KILL QUERY`, PostgreSQL Server-Cursor (`DECLARE … CURSOR`, `FETCH`), SQLite streamt. SQLite-Abfragen aus dem Editor laufen im Arbeitsprozess mit Zeitgrenze. Abbrechen ist eine zweite Anfrage; ein Merker im Cache entscheidet über «abgebrochen», nicht der Fehlercode.

Konsequenzen

- Verbindungsfehler werden je Treiber zugeordnet (MySQL nach Code, PostgreSQL nach Meldungstext).
- Der Arbeitsprozess für SQLite braucht die Warteschlange oder einen Kindprozess; ohne beides gilt die PHP-Laufzeitgrenze.
- Erlaubte Hosts werden vor dem Verbinden geprüft (ADR-004, V-05).

Siehe Solution-Architektur · Feasibility

Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-5)
Betrifft	SchemaEditor, ChangeSet

Kontext

Nichts soll ungesehen passieren (P-02). Die Feasibility belegt: `toSql()` ist auf allen vier genau das, was läuft; `pretend()` liefert für DML eine gute Vorschau, für SQLite-DDL aber eine kaputte. Naiv in einer Transaktion angewendet, wirft Laravel auf MySQL/MariaDB «There is no active transaction» (DDL beendet Transaktionen implizit), und **SQLite löscht beim Umbau einer Elterntabelle alle abhängigen Zeilen** (0 von 2), weil `PRAGMA foreign_keys` in einer Transaktion nicht wirkt.

Optionen

Option	Vorteile	Nachteile
A — alles in einer Transaktion	einfach, scheinbar sicher	auf drei von vier Treibern falsch, auf SQLite Datenverlust
B — Verfahren je Treiber	auf allen vier belegt korrekt (2 von 2 Kindern bleiben)	drei Wege, die Vorschau muss sie erklären
C — Strukturänderungen nur über eigenes SQL	keine Abhängigkeit von Laravel	baut die Grammatiken nach, die SQLite-Umbau schon können

Entscheid

Option B. Vorschau von Strukturänderungen aus `Blueprint::toSql()`, von Datenänderungen aus `pretend()`. Anwenden: MySQL/MariaDB Anweisung für Anweisung ohne Transaktion (Vorschau: «nicht rückrollbar»); PostgreSQL in einer Transaktion; SQLite nach sqlite.org: Fremdschlüssel ausserhalb aus, Umbau in der Transaktion, `foreign_key_check`, Commit, ein. Datenänderungen auf allen vier in einer Transaktion, vorher Sicherung für Rückgängig.

Konsequenzen

- Tests für Strukturänderungen immer mit Elterntabelle und abhängigen Zeilen.
- Bei MySQL/MariaDB meldet die Oberfläche nach einem Fehler, welche Schritte schon gelaufen sind.
- Lektion «SQLite-Tabellenumbau nie in eigener Transaktion» im Vault.

Siehe Solution-Architektur · Feasibility

Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-7)
Betrifft	ReadOnlyGuard, QueryRunner, SQLite-Verbindungen

Kontext

Produktion ist standardmässig schreibgeschützt (V-04), und der SQL-Editor nimmt freies SQL. Ein Schreibschutz über eine Sitzungseinstellung allein lässt sich im Editor abschalten (`SET … READ WRITE`, `PRAGMA query_only = OFF`). Die Feasibility hat einen Wächter mit 5–12 Ausbruchsversuchen je Treiber geprüft: Bestand unverändert.

Optionen

Option	Vorteile	Nachteile
A — nur Lesekonto verlangen	das einzige echte Schloss	viele haben nur ein Konto; lokal unpraktisch
B — Schichten: Lesekonto empfohlen, Nur-lesen-Transaktion je Anweisung, vorbereitet, SQLite nur lesend geöffnet + Authorizer	belegt auf allen vier	jede Anweisung einzeln; Zerleger nötig
C — SQL vor dem Ausführen klassifizieren und Schreibendes ablehnen	ohne Transaktion	Parser-Rennen gegen jede Syntax; Funktionen mit Nebenwirkung

Entscheid

Option B. MySQL/MariaDB: `SET SESSION TRANSACTION READ ONLY` + Transaktion, immer zurückrollen; PostgreSQL: Transaktion + `SET TRANSACTION READ ONLY`, zurückrollen; SQLite: `Pdo\Sqlite::OPEN_READONLY` und `setAuthorizer()` (`ATTACH` nur im erlaubten Ordner). Anweisungen werden auf dem Server zerlegt und einzeln vorbereitet; `unprepared()` ist für Eingaben verboten. Konten mit Superuser- oder Dateirechten werden sichtbar gewarnt (V-10).

Konsequenzen

- Der Zerleger je Dialekt ist Voraussetzung (TB 03, Probe zu Beginn).
- Die Ausbruchsversuche aus `SecurityProbeTest` werden Abnahmetest von V-04.
- PHP 8.5 wird Mindestversion (ADR-011).

Siehe Solution-Architektur · Feasibility

Anhang E — ADR-005 — SQL-Editor: CodeMirror 6

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-4, Freigabe der Pakete durch Hoang)
Betrifft	Oberfläche SQL-Reiter

Kontext

Der Editor braucht Hervorhebung und Vervollständigung je Dialekt aus dem Schema (Q-01). Probeweise installiert: `codemirror` 6.0.2, `@codemirror/lang-sql` 6.10.0 (MIT, 17 Pakete). Belegt: Tabellen aus 1'002 Tabellen in höchstens 2 ms, Aliasse aufgelöst, dialekteigene Schlüsselwörter (`ILIKE` nur PostgreSQL); Chunk 425 KB (138 KB gzip).

Optionen

Option	Vorteile	Nachteile
A — CodeMirror 6	modular, MIT, Dialekte für alle vier, klein genug	eigene Vue-Hülle
B — Monaco	VS-Code-Gefühl	mehrere MB, SQL-Dialekte schwächer, Web-Worker
C — Eigenbau mit Textarea	keine Abhängigkeit	keine Vervollständigung, viel Arbeit

Entscheid

Option A, als eigener Chunk nur auf dem SQL-Reiter geladen.

Konsequenzen

- Sichtprüfung im Browser durch Hoang steht aus (Probe-Seite `/probe/editor`).
- Bei Updates der Pakete Lizenzliste neu erzeugen (ADR-010).

Siehe Solution-Architektur · Feasibility

Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel

Feld	Wert
Status	Angenommen — bestätigt von Hoang am 28.10.2026 (Endlos scrollen statt Seiten zu 50)
Datum	28.10.2026 (Foundation; F-3)
Betrifft	Datenraster

Kontext

Tabellen mit Millionen Zeilen müssen sofort erscheinen (N-01). Gemessen: erste Seite über den Schlüssel 0,1–0,6 ms in der Datenbank, `OFFSET 900000` 18–232 ms; im Browser 281 ms für die erste Seite, davon fast alles Laravel und Herd; im DOM bleiben rund 25 Zeilen.

Optionen

Option	Vorteile	Nachteile
A — eigenes Raster mit `@tanstack/vue-virtual` (MIT, 34 KB Chunk)	volle Kontrolle über Bearbeiten, Zustände, Gestaltung	Raster selbst bauen
B — fertiges Datenraster (AG Grid Community u. a.)	viel fertig	schwer, eigene Bedienlogik, Gestaltung schwer anzugleichen
C — ohne Virtualisierung, Seiten zu 100	einfach	kein flüssiges Scrollen, Seitenwechsel

Entscheid

Option A. Seiten zu 200 Zeilen über den Schlüssel, die nächste vorgeladen; `OFFSET` nur ohne Primärschlüssel; erste Spalte fixiert, waagrecht scrollen nur hier.

Konsequenzen

- Geänderte, neue und zu löschende Zeilen sind Zustand im Browser bis «Anwenden» (D-04).
- Kartenliste unter 768 px (G-07).

Siehe Solution-Architektur · Feasibility

Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-8, F-3)
Betrifft	Transfer

Kontext

Gemessen bei einer Million Zeilen: `lazyById(5000)` 3–3,3 MB auf allen vier; `cursor()` gepuffert 62 MB (MySQL/MariaDB), PostgreSQL +105 MB im Prozess. Import von 100'000 Zeilen in Paketen zu 1'000: 143–502 ms. Nach dem Füllen fehlt MySQL/MariaDB die Statistik — Sortieren über den Index 300 statt 2 ms.

Optionen

Option	Vorteile	Nachteile
A — Pakete über den Schlüssel, im Hintergrund, Datei mit Frist	gleich auf allen vier, wenig Speicher	Warteschlange nötig (ohne Worker synchron mit Grenze)
B — Server-Werkzeuge (`mysqldump`, `pg_dump`)	schnell, vollständig	nicht überall installiert, Versionen müssen passen — nur für Sicherung (E-03)

Entscheid

Option A für Export und Import; Probelauf beim Import in einer zurückgerollten Transaktion; nach eigenen Importen `ANALYZE` (E-06). Server-Werkzeuge nur für Sicherung und Wiederherstellung (E-03, v1.1).

Konsequenzen

- Exportdateien unter `storage/app/private/transfers`, Frist 24 Stunden.
- Tabellen ohne Primärschlüssel exportieren über `cursor()` ungepuffert bzw. Server-Cursor.

Siehe Solution-Architektur · Feasibility

Anhang H — ADR-008 — Anmeldung ohne Passwort

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Vorgabe Hoang; F-6)
Betrifft	Anmeldung, Konten

Kontext

Vorgabe: kein Passwort, nur Passkey und Code per Mail; kein Dashboard, kein Profil. Fortify 1.41 registriert `POST /login` und die Passwortbestätigung unabhängig von den Features; das Feature `passkeys` steht im Starter-Kit mit `confirmPassword: true`. Basalto hat dasselbe mit denselben Versionen gebaut (Code-Login, Abweisung, Software-Authenticator für Tests).

Optionen

Option	Vorteile	Nachteile
A — Fortify nur für Passkeys, eigener Code-Login, Passwort-Endpunkte abweisen	Muster erprobt, Passkeys von Laravel	Fortify-Updates können Routen verschieben (Lektion)
B — Fortify ganz entfernen, Passkeys selbst	keine Überraschungen aus Fortify	WebAuthn selbst prüfen — mehr Risiko

Entscheid

Option A. `Fortify::authenticateUsing()` liefert `null`; Passwortbestätigung durch Bestätigung per Passkey bzw. Code ersetzt; Passwort-Spalten und -Tabellen entfernt; erstes Konto per `phpdbadmin:install`, weitere per Einladung.

Konsequenzen

- Test ruft jede alte Adresse des Starter-Kits auf (404/405).
- Vor jedem `composer update` Routenliste sichern und vergleichen.
- Parameter nie `{passkey}` nennen (globale Bindung von `laravel/passkeys`).

Nachtrag TB 00 (28.10.2026)

Umgesetzt mit einer strengeren Form von Option A: `Fortify::ignoreRoutes()` statt `authenticateUsing()`. Fortify registriert damit **keine** eigenen Routen; `routes/auth.php` trägt die Passkey-Wege der Bibliothek (`Laravel\Passkeys\Http\Controllers`) unter den Namen, die Fortify verwendet hätte. Ein Passwort-Endpunkt existiert gar nicht (`POST /login` → 405). Die Middleware

`password.confirm` leitet auf eine eigene Seite (Bestätigen per Passkey oder Code an die eigene Adresse); jede Anmeldung setzt die Bestätigung. Fortify bleibt nur, weil es die Passkey-Konfiguration (`passkeys.*`) aus `config/fortify.php` ableitet.

Ersteinrichtung (28.10.2026, Wunsch Hoang): Das erste Konto entsteht im Browser oder per Konsole. Im Browser schützt ein Einrichtungsschlüssel, den die App beim ersten Aufruf von `/setup` als Datei auf dem Server ablegt (`storage/app/private/einrichtungsschluessel.txt`, 0600) und nach der Einrichtung löscht — so wird nicht der erste Besucher einer frisch installierten Seite Administrator (T-19). Bewusst nicht gewählt: offene Einrichtung für den ersten Besucher (Übernahme), Freigabe nach Absender-IP (hinter einem Reverse-Proxy kommt jede Anfrage von `127.0.0.1`), Bestätigung per Mail (auf einem frischen Server oft noch nicht eingerichtet). Nach der Einrichtung folgt das Onboarding mit Passkey.

Siehe Solution-Architektur · Feasibility

Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; R-31)
Betrifft	Installation, Betrieb

Kontext

Die App-Datenbank hält Konten, Verbindungen, Verlauf und Protokoll — wenig Daten, wenige gleichzeitige Schreiber. Eine Installation neben fremden Datenbanken soll ohne zusätzlichen Server laufen.

Optionen

Option	Vorteile	Nachteile
A — SQLite als Vorgabe, die anderen drei möglich	keine Einrichtung, Sicherung = eine Datei	Schreibsperren bei vielen Parallelzugriffen (`busy_timeout`, WAL)
B — MariaDB/PostgreSQL vorgeschrieben	robuster bei vielen Konten	zusätzlicher Server nur für phpdbadmin

Entscheid

Option A, mit WAL und `busy_timeout`; die Testsuite läuft mit jeder der vier als App-Datenbank (B-06).

Konsequenzen

- Regeln für Spaltentypen gelten trotzdem für alle vier (Datenmodell § 1).
- Die App-Datenbank erscheint nie als Zielverbindung (R-17).

Siehe Solution-Architektur · Feasibility

Anhang J — ADR-010 — Lizenz MIT

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Entscheid Hoang, R-40; F-9)
Betrifft	Repository, Abhängigkeiten

Kontext

Hoang hat MIT gewählt (Markt überwiegend kostenlos, Schaufenster für Nguyen Development). Geprüft: Composer 101 Pakete MIT/BSD/Apache, `nette/*` wahlweise BSD-3; npm-Laufzeit MIT/ISC/BSD/Apache; neue Pakete MIT.

Optionen

Option	Vorteile	Nachteile
A — MIT	einfach, verbreitet	kein Schutz gegen geschlossene Ableitungen
B — GPL-3	Ableitungen bleiben offen	schreckt Einsatz in Firmen ab

Entscheid

Option A. `LICENSE` mit MIT vor dem ersten öffentlichen Push; Code aus GPL-Projekten (phpMyAdmin, phpPgAdmin, Adminer) wird nicht übernommen.

Konsequenzen

- Lizenzliste vor jedem Release erneut erzeugen.
- Hinweis «nicht verbunden mit phpMyAdmin» in README und Website (F-10).

Siehe Solution-Architektur · Feasibility

Anhang K — ADR-011 — PHP 8.5 als Mindestversion

Feld	Wert
Status	Angenommen (Hoang, 28.10.2026)
Datum	28.10.2026 (Foundation)
Betrifft	Installation, Hosting, `composer.json`

Kontext

Der Schutz von SQLite gegen `ATTACH` fremder Dateien braucht `Pdo\Sqlite::setAuthorizer()`. Geprüft in Herd: PHP 8.4.27 hat `Pdo\Sqlite`, aber keinen Authorizer; PHP 8.5.12 hat ihn. Laravel 13 verlangt nur PHP 8.3, `composer.json` steht auf `^8.3`.

Optionen

Option	Vorteile	Nachteile
A — PHP 8.5 vorschreiben	Schutz vollständig, ein Weg	Hosting mit PHP 8.4 fällt weg
B — PHP 8.4 erlauben, SQLite-`ATTACH` dort per Textprüfung sperren	breiter einsetzbar	Textprüfung ist schwächer; zwei Wege testen
C — PHP 8.4 erlauben, SQLite-Verbindungen dort nur lesend und ohne Editor	einfach	halbe Funktion

Entscheid

Option A (entschieden von Hoang am 28.10.2026). `composer.json` auf `^8.5` in TB 00; Herd läuft bereits mit 8.5.

Konsequenzen

- Die Installationsanleitung nennt PHP 8.5 und `pdo_sqlite`, `pdo_mysql`, `pdo_pgsql`.

Siehe Solution-Architektur · Feasibility

Anhang L — Timebox 00 — Fundament

Stand: 28.10.2026 · ✅ abgeschlossen — gestartet auf Hoangs «Ja, TB 00 starten», gebaut in fünf Scheiben nach dem Timebox-Plan § 2; während Scheibe 5 auf Hoangs Wunsch ergänzt um **Ersteinrichtung und Passkey-Onboarding**.

Ziel

Ein sauberes Fundament ohne Starter-Kit-Reste: Anmeldung ohne Passwort, Verbindungen zu vier Datenbanken anlegen und testen, eine Dialekt-Schicht mit Vertragstests — geprüft gegen MySQL 8.4, MariaDB 10.11, PostgreSQL 18 und SQLite.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Starter-Kit bereinigen	A-01, A-04, A-05, B-01	✅
2 Anmeldung	A-02, A-03, A-06, A-09	✅ (A-03: Handtest mit Touch ID offen, Hoang)
3 Verbindungen	V-01, V-02, V-03, V-05, B-06	✅
4 Dialekt-Schicht	B-02 (+ Grundlage V-04, V-10)	✅
5 Qualität und Abnahme	B-03, B-08	✅ lokal; Workflow noch nie auf GitHub gelaufen (kein Remote)
Zusatz: Ersteinrichtung	A-06, A-03	✅ (Wunsch Hoang während Scheibe 5)

Wireframe-Bereiche 1 (Anmeldung), 2 (Code), 3 (Verbindungen), 4 (Verbindung anlegen), 13 (Konto).

Was entstanden ist

Anmeldung ohne Passwort (ADR-008, Nachtrag). Fortify registriert keine eigenen Routen mehr (`Fortify::ignoreRoutes()`); `routes/auth.php` trägt nur die Passkey-Wege der Bibliothek. `POST /login` antwortet 405 — einen Passwort-Endpunkt gibt es nicht. Code per Mail: sechs Ziffern, HMAC-Prüfsummen für Code, Adresse und IP, zehn Minuten, einmalig (bedingtes Update gegen zwei gleichzeitige Einlösungen), Sperre nach fünf Fehlversuchen, benannte Limiter je Adresse und IP, gleiche Antwort für unbekannte Adressen, die Mail geht erst nach der Antwort hinaus (`defer`), die Sitzung wird neu erzeugt. Der Code steht bewusst nicht im Betreff (Sperrbildschirm). Die Passwortbestätigung ist ersetzt durch «Bestätigen» per Passkey oder Code an die eigene Adresse; jede Anmeldung gilt als Bestätigung. Passkeys anlegen, umbenennen, entfernen im Konto. Protokoll: Anmeldungen (mit Weg: Code, Passkey, Einladung, Einrichtung), Fehlversuche (Adresse nur als Prüfsumme), Abmelden, Passkey-Änderungen.

Ersteinrichtung und Onboarding. Ohne jedes Konto zeigt `/login` die Ersteinrichtung statt einer Anmeldung ohne Ausweg. **Im Browser** (Hauptweg, Hoangs zweiter Wunsch: «nicht jeder hat ein Terminal»): `/setup` legt einen Einrichtungsschlüssel als Datei auf dem Server ab (`storage/app/private/einrichtungsschluessel.txt`, 0600, nie überschrieben, nach der Einrichtung gelöscht); wer die Installation hochgeladen hat, liest ihn über Dateimanager, FTP oder Editor und gibt ihn mit Name und E-Mail ein — ein zufälliger Besucher kann so nicht Administrator werden (T-19). **Per Konsole** (Alternative): `php artisan phpdbadmin:install` legt genau ein Administratorkonto an und gibt

einen signierten Link aus (30 Minuten, gilt nur bis zur ersten Anmeldung; erst ein Knopf meldet an, damit Link-Vorschauen ihn nicht verbrauchen). Die erste Anmeldung gelingt so auch, wenn der Server noch keine Mail versenden kann; solange die Ersteinrichtung offen ist, stellt der Befehl einen neuen Link aus. Wer sich ohne Passkey anmeldet (Code, Einladung, Einrichtungslink), landet im Onboarding «Passkey jetzt anlegen»; das erste Konto sieht die Variante «Ersteinrichtung» mit den nächsten Schritten. A-06 ist in CR-001 Fassung 5 entsprechend angepasst.

Verbindungen. Tabellen `connections` (ULID, Passwort mit `encrypted`-Cast) und `connection_user` nach dem Datenmodell. Anlegen, bearbeiten, entfernen für vier Treiber; Produktion schreibgeschützt vorgegeben; Passwort nie im Browser (die Seite erfährt nur, ob eines gespeichert ist), leeres Feld behält das gespeicherte, «nicht speichern» möglich. «Verbindung testen» mit Prüfschritten (Host erlaubt → Server erreichbar → Anmeldung → Rechte des Kontos), Fehler in Klartext je Treiber, Meldung des Servers zum Aufklappen. `HostPolicy`: Namen, Platzhalter, Adressen, Netze; einmal auflösen und zur geprüften Adresse verbinden; SQLite nur in erlaubten Ordnern über `realpath()`, die App-Datenbank nie. `{connection}` löst nur sichtbare Verbindungen auf — fremde sind 404 (T-16).

Dialekt-Schicht (B-02). Vertrag `App\Dialects\Dialect` mit Basis über Laravels Introspektion (Schema immer ausdrücklich) und vier Umsetzungen: Konfiguration (UTC, Zeitlimit beim Verbinden, SSL, Schreibschutz beim Öffnen bzw. für die Sitzung), Version, Fehlerzuordnung, Schemas, Tabellen und Sichten mit Zeilenschätzung, Check-Constraints und Trigger, Zeitlimit je Anweisung, Schreibschutz-Wächter (eine Anweisung je Aufruf in einer Nur-lesen-Transaktion, immer zurückgerollt), Kontorechte. `ConnectionFactory`: keine Mehrfachanweisungen, kein `LOCAL INFILE`, SQLite-`ATTACH` nur in erlaubten Ordnern (Authorizer).

Gestaltung. Palette aus `werkzeuge/kontraste.py` im Stylesheet (hell und dunkel), Inter und JetBrains Mono beim Build heruntergeladen und mit ausgeliefert (G-02), Wortmarke (G-06), Umgebungszeichen ○ ◐ ● mit Wort (G-03).

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
A-01	alte Adressen 404/405, Spalten weg	`StarterKitRemovalTest`: 17 Adressen 404, `POST /login` 405, keine Spalten `password`, `two_factor_*`, keine Tabelle `password_reset_tokens`
A-02	Erfolg, Ablauf, Wiederverwendung, falscher Code, Drosselung, unbekannte Adresse, Sitzung neu	`LoginCodesTest` (6), `LoginControllerTest` (14) — Sitzungs-ID vor/nach über das Cookie
A-03	Zeremonie mit virtuellem Authenticator; einmal Touch ID	`PasskeyLoginControllerTest`, `PasskeyManagementTest`, `ConfirmControllerTest` mit dem Software-Authenticator (aus Basalto); **Touch ID von Hand: offen (Hoang)**
A-04	`/dashboard` 404, Anmeldung führt zur Übersicht	`StarterKitRemovalTest`; Anmeldung mit Passkey → Übersicht, ohne Passkey → Onboarding
A-05	`/settings/profile` 404, Kontomenü	Test + Browser-Durchlauf (Menü Sicherheit, Darstellung, Abmelden)
A-06	Einladung annehmen, abgelaufene ablehnen; Befehl legt genau ein Admin-Konto an; falscher Schlüssel legt kein Konto an	`InvitationControllerTest` (7), `InstallCommandTest` (5), `InviteCommandTest` (3), `SetupControllerTest` (5), `FirstSetupControllerTest` (6), `SetupKeyTest` (4); im Browser mit falschem und richtigem Schlüssel durchgespielt

Anforderung	Kriterium (CR-001)	Nachweis
A-09	Eintrag mit Zeit, Konto bzw. Adresse (gehasht), IP-Prüfsumme	Tests prüfen `auth.login`, `auth.failed` (ohne Klartext-Adresse), `auth.logout`
V-01	je Treiber ein Test gegen den echten Dienst; Fehler am Feld	`ConnectionTesterTest` (Herd-Dienste), `ConnectionControllerTest`
V-02	je Fehlerart ein Test mit erzwungenem Fehler	nicht erreichbar, Zugang verweigert (ohne Passwort in der Meldung), Datenbank fehlt — je Server; SSL (PostgreSQL), Host nicht erlaubt, Datei keine Datenbank
V-03	kein Passwort in Props und JSON; Spalte verschlüsselt	Übersicht, Bearbeiten, Test-Endpunkt; Rohwert ≠ Klartext
V-05	nicht erlaubter Host und Pfad (auch `..`, Verknüpfung)	`HostPolicyTest` (21 Fälle aus 5 Tests), Controller am Feld; im Browser `~/Development/../../../etc/hosts` abgewiesen
B-01	Routen und Seiten ohne Starter-Kit-Reste	Routenliste: nur eigene und Passkey-Wege; Seitenordner `auth`, `connections`, `account`, `onboarding`, `probe`
B-02	Vertragstest gegen alle vier Umsetzungen	`DialectContractTest`: 12 Tests, 32 Fälle über die Treiber — alle grün
B-03	lokales Skript fährt alle vier; Workflow sparsam	`composer ci:local`; Workflow mit `paths-ignore`, `concurrency`, `timeout-minutes`, Matrix wöchentlich — **noch nie auf GitHub gelaufen**
B-06	Suite läuft mit jeder App-Datenbank	`composer test:app-db`: grün mit SQLite, MariaDB, MySQL, PostgreSQL als App-Datenbank
B-08	Prüfkette in der Reihenfolge der CI	`vp check` → `vue-tsc` → Pint → Larastan (Stufe 7) → PHPUnit

Stand der Suite: 221 Tests, 209 bestanden, 12 übersprungen (die schweren Raster- und Export-Proben mit 1 Mio. Zeilen, bewusst nur auf Wunsch).

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
01 Anmeldung	Sprachwahl Deutsch · English fehlt	Englisch ist nicht Teil von TB 00
01 Anmeldung	neu: Ersteinrichtung, wenn es kein Konto gibt	Wunsch Hoang; noch nicht im Generator
02 Code	«Code abgelaufen» hat keine eigene Ansicht; eine Meldung für falsch und abgelaufen	verrät nichts über den Zustand eines Codes
02 Code	Hinweis ergänzt: «…sofern es dazu ein Konto gibt»	ehrlich bei gleicher Antwort für unbekannte Adressen
03 Verbindungen	ohne «Zuletzt geöffnet», Favoriten, Befehlspalette, Klick in den Arbeitsbildschirm	kommen mit ihren Timeboxes (V-06, TB 01, TB 11)
04 Verbindung anlegen	SSL ohne «Zertifikat prüfen»	verbunden wird zur geprüften Adresse (V-05); Prüfen des Zertifikats braucht eine eigene Anforderung
13 Konto	ohne Sitzungen und Dichte	A-08 (TB 04), G-04 mit dem Raster (TB 01)
13 Konto	Kontomenü ohne «Verwaltung»	TB 07
neu	Ersteinrichtung (Link) und Onboarding «Passkey jetzt anlegen»	Wunsch Hoang; Wireframes dazu noch nicht im Generator

Browser-Durchlauf

Gegen einen gebauten Stand in einem Herd-Workspace (der eingebaute Browser erreicht den Vite-Dev-Server nicht): Bereiche 1–4 und 13 bei 1440 und 375 px, hell und dunkel, Anmeldung per Code mit echter Mail in Herd, Verbindung zu MariaDB anlegen und testen, abgewiesener Pfad mit `..`, Ersteinrichtung auf einer Installation ohne Konto. Gefunden und behoben: Adressen brachen mitten im Wort, Codefeld zu klein, Hinweis ohne Mail-Symbol und Ablaufzeit, Kopf «Verbindung testen» brach um, **Warnsymbol fehlte wegen eines vergessenen Imports** (jetzt meldet `vue-tsc` unbekannte Komponenten), «· Konto» ohne Datenbank, SQLite-Platzhalter mit einem hier nicht erlaubten Ordner, leeres Passwort galt als «gespeichert».

Entscheide in der Timebox

Entscheid	Begründung
`Fortify::ignoreRoutes()` statt `authenticateUsing()`	kein Passwort-Endpunkt statt eines abweisenden (ADR-008, Nachtrag)
Route `login.code.show` statt `login.code.form`	Wayfinder hängt verschachtelte Routen an die Funktion; `form` überschrieb die Formular-Variante und machte das Formular zum GET
kein «angemeldet bleiben» (`remember_token` entfernt)	Datenmodell § 1; Sitzungsdauer genügt
Probe-Seiten bleiben lokal hinter der Anmeldung	Sichtprüfung des Editors durch Hoang offen; Raster (TB 01) und SQL-Editor (TB 05) ersetzen sie
Proben F-1, F-7 und Teile von F-2 → Vertragstests	Probe-Dateien entfernt; Abbrechen, DDL-Vorschau, Raster und Export bleiben Proben bis zu ihren Timeboxes
Lesekonto `phpdbadmin_lesen` und Datenbanken `phpdbadmin_test` , `phpdbadmin_app_test` auf den lokalen Herd-Servern	Vertragstest «warnt nicht» (V-10) und Suite mit App-Datenbank auf den Servern

Commits

`083e3c9` Scheiben 1+2 · `b04de27` Scheibe 3 · `ceab0e8` Scheibe 4 · `2bea65c` Scheibe 5 (Prüfskript, Workflow, Funde) · `95aa78c` Ersteinrichtung und Onboarding · `7336538` Protokoll · `bfb091c` symfony/mime · `fc75729` Ersteinrichtung im Browser.

Offen

- **Touch ID von Hand** (A-03): einmal einen Passkey auf dem Mac anlegen und damit anmelden — Hoang.
- **Workflow auf GitHub:** noch kein Remote; der erste Push zeigt, ob beide Aufträge laufen.
- ~~**Sicherheitshinweis** `symfony/mime` (CVE-2026-45070, mittel)~~ — erledigt 28.10.2026 auf Hoangs Freigabe: 8.1.7 → 8.1.10, `composer audit` ohne Hinweise, Prüfkette auf allen vier App-Datenbanken grün, Testmail über Herd mit Umlauten im Betreff angekommen.
- Wireframes für Ersteinrichtung und Onboarding im Generator nachziehen — in Arbeit auf einem eigenen Zweig; dort fehlt noch die Einrichtung im Browser mit Einrichtungsschlüssel (`fc75729`).
- Sichtprüfung der Gestaltung durch Hoang am eigenen Bildschirm (G-01 bis G-08).

Gelernt

- Textersetzungen per Skript immer mit Prüfung, ob die Stelle gefunden wurde — der Formatierer bricht Zeilen um, und eine stille Nicht-Ersetzung liess ein Symbol verschwinden (Lektion im Vault).
- `vue-tsc` meldet unbekannte Komponenten erst mit `checkUnknownComponents` (Lektion im Vault).
- Wayfinder: Routennamen nie auf `.form` enden lassen (Lektion im Vault).
- Der eingebaute Browser erreicht den Vite-Dev-Server nicht; für Sichtprüfungen einen Herd-Workspace mit Build nutzen und dessen `artisan dev` anhalten (Projekt-Memory).

Anhang M — Timebox 01 — Katalog und Navigator

Stand: 29.10.2026 · ✅ abgeschlossen bis auf V-06 (Favoriten, «zuletzt geöffnet»), das mit TB 07 im selben Zweig gebaut wird, weil beides an `connection_user` hängt. Gestartet auf Hoangs «nächste tb»; ab dem Abend auf seinen Auftrag «mache alle TB durch» ohne Rückfragen weitergeführt.

Ziel

Eine Verbindung öffnen und darin arbeiten können: Arbeitsbildschirm mit Navigator für alle vier Datenbanken, Struktur einer Tabelle in denselben Begriffen, Schreibschutz und Umgebung sichtbar, Konten mit zu vielen Rechten gewarnt — auf Deutsch und Englisch.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Arbeitsbildschirm öffnen	V-04, V-10	✅ Kopf mit Umgebung (Farbe, Zeichen, Wort), roter Streifen bei Produktion, Titel «PROD · …», Kontowarnung; Passwort je Sitzung, Fehlerkarte statt Fehlerseite
2 Navigator	O-01	✅ Datenbank → Schema → Gruppen (Tabellen, Sichten, Routinen, Trigger, Ereignisse, Sequenzen) mit Filter «/»; 1'000 Tabellen in einem Test
Zusatz «Alle Datenbanken anzeigen»	V-01	✅ Wunsch Hoang; Fehler aus seinem Durchlauf (Endlosschleife) behoben
3 Strukturansicht	S-01, O-04	✅ Wireframe 08 bei 1440 und 375 px
4 Favoriten, zuletzt geöffnet	V-06	⏳ im Zweig von TB 07
5 Deutsch und Englisch	N-06, O-06	✅ alle Texte über das Sprachsystem, Test über alle Schlüssel
6 Abnahme	N-04, O-04	✅ O-04 Bildschirmfotos; N-04 Untergrenzen in der Wochen-Matrix (noch nie gelaufen, kein Remote)

Wireframe-Bereiche 3 (Verbindungen), 5 (Arbeitsbildschirm), 8 (Struktur).

Was entstanden ist

Arbeitsbildschirm (`/c/{verbindung}`, `/c/{verbindung}/s/{schema}`). `TargetConnections` öffnet Zielverbindungen je Anfrage mit Zeitlimit (15 s) und Schreibschutz, wenn die Verbindung schreibgeschützt ist; jede lesende Anweisung läuft zusätzlich im Wächter des Dialekts. Fehler der Zielverbindung werden zur Fehlerkarte mit Ursache und nächstem Schritt, fehlende Passwörter («nicht speichern») fragt eine eigene Seite einmal je Sitzung ab. Fehler der eigenen App-Datenbank bleiben Fehler der App (früher als «Verbindung gescheitert» getarnt — im Durchlauf von TB 02 gefunden).

Alle Datenbanken (Wunsch Hoang). Ohne Datenbanknamen zeigt MySQL/MariaDB alle Datenbanken als Schemas, PostgreSQL öffnet `postgres` und bietet die übrigen Datenbanken im Navigator an (`?db=`). Der Fehler aus Hoangs Durchlauf («Maximum call stack size … Infinite recursion») lag in Laravels `DatabaseManager`: zwei Zielverbindungen mit gleichem Namen trennten sich gegenseitig, der Neuaufbau

traf das falsche Objekt. Zielverbindungen stehen seither in keinem globalen Register; der Neuverbinder erneuert genau dieses Objekt und richtet Schreibschutz, Authorizer und Zeitlimit wieder ein. Gegenprobe: Auf dem alten Stand hängen die neuen Tests in der Schleife.

Struktur (`…/o/{objekt}/structure`). `Structure` liest Spalten mit Typ im Original und einer **Art**, die in allen vier Datenbanken gleich heisst (`ColumnKind`: Zahl, Text, Auswahl, Wahrheitswert, Datum und Zeit, JSON, Binär, Kennung, Geometrie, Liste, Sonstiges), Indizes, Fremdschlüssel in beide Richtungen, Checks, Trigger und DDL. Gleichmacher für O-04: MariaDB-JSON (Alias für `longtext` mit `json_valid`) gilt als JSON, eigene Aufzählungstypen in PostgreSQL als Auswahl, Vorgaben als SQL-Literal (`'offen'` statt `offen` bzw. `'offen'::character varying`), MariaDBs Text «NULL» als keine Vorgabe.

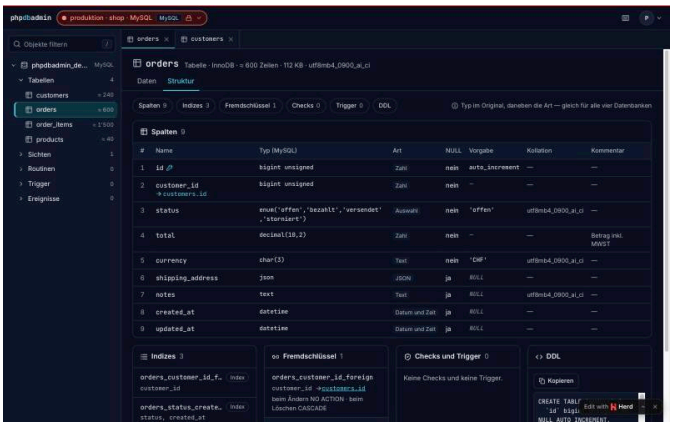
Deutsch und Englisch (N-06, O-06). Der deutsche Text ist der Schlüssel im Code (`__('…')`, `t('…')`, `tc('…')`), `lang/en.json` liefert Englisch (britisch). Sprache aus dem Konto, sonst aus dem Browser, sonst Deutsch; Mails in der Sprache des Kontos. Die Oberfläche bekommt die Übersetzungen einmal je Sprache (Once-Prop). Sprachwahl im Konto unter «Darstellung». Alle bestehenden Seiten hat ein Helfer in einer eigenen Arbeitskopie umgestellt (Zweig zusammengeführt, 322 Zeilen ohne Abweichung).

Abnahmekriterien

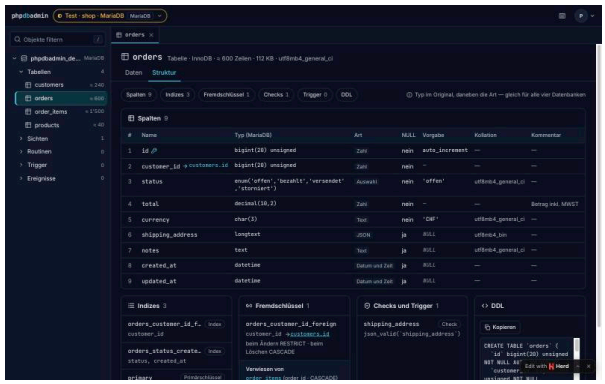
Anforderung	Kriterium (CR-001)	Nachweis
V-04	schreibgeschützte Verbindung lehnt jede schreibende Anweisung ab, auch Ausbruchsversuche	`DialectContractTest` (Wächter je Treiber), `TargetConnectionsTest` 19 Tests: Schreibschutz übersteht einen Neuaufbau (Laravel verbindet bei SQLSTATE 25006 neu)
V-10	Herd-`root` warnt, Lesekonto nicht	`DialectContractTest` gegen root und `phpdbadmin_lesen` auf MySQL, MariaDB, PostgreSQL
O-01	1'000 Tabellen flüssig	`WorkspaceControllerTest::test_navigator_with_a_thousand_tables_stays_fast` (unter 1 s inkl. Seite)
O-04	dieselbe Tabelle in allen vier Datenbanken	Bildschirmfotos unten; `StructureTest` 17 Tests (Art, Vorgaben, Fremdschlüssel beidseitig je Treiber)
S-01	Beispieltabelle mit allen Merkmalen je Treiber	`StructureTest` (Kommentar, Aufzählung, JSON, Standardwerte, eindeutiger Index, Check, Trigger, Sicht), `ObjectControllerTest`
N-06 / O-06	Sprachdateien vollständig ohne Rückfall	`LanguageFilesTest` (5): jeder Schlüssel mit englischer Zeile, keine verwaisten, gleiche Platzhalter, keine dynamischen Schlüssel, Feldnamen beider Sprachen; `LocaleControllerTest` (5)
N-04	Untergrenzen	Wochen-Matrix «untergrenzen» (MariaDB 10.6, MySQL 8.0, PostgreSQL 14) — **noch nie gelaufen** (kein Remote); lokal nur aktuelle Versionen

O-04: dieselbe Tabelle `orders` in allen vier Datenbanken

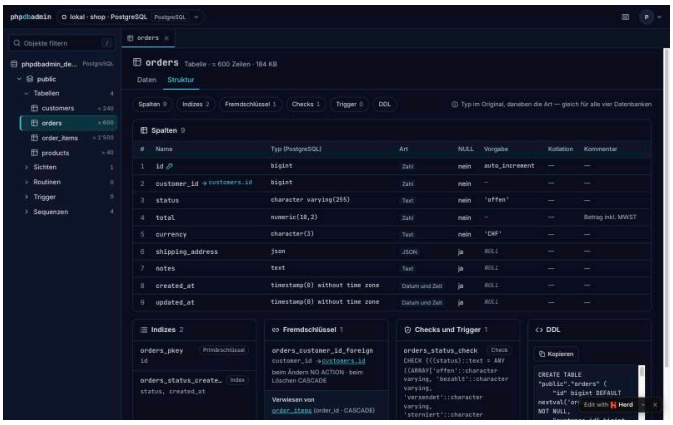
MySQL 8.4 (Produktion, schreibgeschützt)



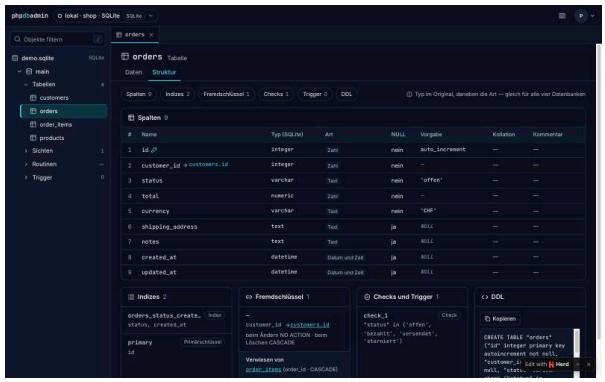
MariaDB 10.11 (Test)



PostgreSQL 18 (lokal)



SQLite (lokal)



Gleich: Aufbau, Begriffe, Art je Spalte, Vorgaben als Literal, Fremdschlüssel beidseitig. Erklärte Unterschiede: PostgreSQL legt Laravels `enum()` als `varchar` mit Check an (Art «Text», der Check steht daneben), SQLite kennt weder Zeilenschätzung noch Kollation noch Namen für Fremdschlüssel.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
08 Kopf	Grösse als eine Zahl statt «Daten · Indizes»	Laravel liefert die Gesamtgrösse; getrennte Werte bräuchten je Treiber eigene Abfragen — später, falls gewünscht
08 Knöpfe	«Tabelle ändern», «Exportieren», «Im Editor öffnen» fehlen	kommen mit TB 05, TB 06, TB 03
08 Reiter	«Beziehungen» und «SQL» fehlen	Beziehungsdiagramm S-07 (TB 10), SQL-Editor (TB 03)
05 Kopf	Suchfeld ⌘K fehlt	Befehlspalette O-02 (TB 03)

Tests

Neu oder erweitert: `StructureTest` (17), `ColumnKindTest` (25), `ObjectControllerTest` (Struktur-Teil), `TargetConnectionsTest` (+3, unabhängige Verbindungen, Schlüssel je Datenbank), `WorkspaceControllerTest` (+1, Ablauf aus Hoangs Durchlauf), `DialectContractTest` (+1, VACUUM trotz

ATTACH-Wächter), `LanguageFilesTest` (5), `LocaleControllerTest` (5) und die Englisch-Tests des Übersetzungs-Helfers. Zielserver-Tests laufen über `PHPDBADMIN_TEST_DATABASE` auch parallel in mehreren Arbeitskopien.

Commits

`d4766fa` Scheiben 1+2 · `0842694` Dialekt · `777ce66` alle Datenbanken · `a35ad37` Endlosschleife · `3dd2e9b` Struktur lesen · `884bf8e` Testdatenbank einstellbar · `dcf9e21` Sprachgrundlage · `6db5d35` Strukturansicht · `6a72d92` · `d2227f5` VACUUM · `558b4ae` Vorgaben, Chip · `c503ced` – `7f4c944` Sprache (Helfer) · `80fed1d` Merge · `8069824` Untergrenzen · `1c8f969` MariaDB-Vorgabe.

Gelernt

- Laravels `DB::build()` mit gleichem Namen ersetzt die vorhandene Verbindung; `DB::reconnect()` erneuert das Objekt im Register, nicht das in der Hand. Zielverbindungen deshalb ausserhalb des Registers bauen (Lektion im Vault).
- «Connection refused» hält Laravel für eine verlorene Verbindung und versucht neu — ohne Neuverbinder ein `LostConnectionException`. Sofort verbinden, dann kommt der Fehler als PDO-Fehler.
- Ein leeres PHP-Array wird zu JSON `[]`, und `[].sort` ist in JavaScript eine Methode: Zuordnungen an die Oberfläche als Objekt senden.
- Symfony schickt in Tests `Accept-Language: en-us` mit — mit Sprachwahl aus dem Browser kippen sonst alle deutschen Erwartungen.

Anhang N — Timebox 02 — Datenraster und Filter

Stand: 29.10.2026 · ✅ abgeschlossen — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen gebaut.

Ziel

Die Daten jeder Tabelle und Sicht schnell und in allen vier Datenbanken gleich sehen: blättern, sortieren, filtern, suchen, zählen, Fremdschlüsseln folgen — auch bei einer Million Zeilen.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Datendienst	D-01, D-02, D-03, D-06	✅ `TableData`, `RowDetail`, `Cells` — 52 Tests über vier Treiber
2 Raster und Zeilen-Detail	D-01–D-03, D-06, O-09	✅ Wireframes 05 und 07 (Detail lesend; Bearbeiten mit TB 04)
3 Reiter und Kürzel	O-03, O-05	✅
Messung	N-01	✅ unter 20 ms für die erste Seite

Was entstanden ist

Datendienst. Blättern über den Primärschlüssel (zusammengesetzt ausgeschrieben, weil nicht jede Version Zeilenwerte vergleicht), sonst über den Versatz; der Primärschlüssel ist immer letztes Sortierkriterium. **NULL steht in beiden Richtungen zuletzt** — ausdrücklich über `case when … is null`, nicht nach Gewohnheit des Treibers. Geführte Filter (=, ≠, <, ≤, >, ≥, enthält, enthält nicht, beginnt, endet, ist NULL, ist nicht NULL) mit und/oder; alle Werte gebunden, `%` und `_` im Wert als Zeichen (Escape `!`). «enthält» ohne Gross-/Kleinschreibung: PostgreSQL `ILIKE` auf den Text der Spalte, MySQL/MariaDB nach Kollation (meist auch ohne Akzente), SQLite über eine Unicode-Faltfunktion (`phpdbadmin_fold`), die die `ConnectionFactory` registriert — sonst fände «MÜLLER» nicht «Müller». Freies WHERE als Klammerausdruck im Schreibschutz-Wächter mit Zeitlimit; Suche über alle Textspalten. Zählen genau bis `PHPDBADMIN_EXACT_COUNT_LIMIT` (50'000), darüber die Schätzung des Servers, gefiltert auf grossen Tabellen offen mit «genau zählen». Werte gleich in allen vier: Wahrheitswerte `true`/`false` (MySQL liefert 1/0), Binärdaten als Grösse und Anfang, lange Texte gekürzt, Ganzzahlen über 2^53 als Text, PostgreSQL-`bytea` aus dem Strom gelesen. Fremdschlüssel bekommen die Beschriftung der Zielzeile (bevorzugt `name`, `title`, `email` …). `TrustedSql` kennzeichnet die zwei Stellen mit absichtlich rohem SQL.

Raster (`/c/{verbindung}/s/{schema}/o/{objekt}`). Erste Seite kommt mit der Seite, weitere beim Scrollen (`rows`), dazu `count`, `row` und `view`. Virtualisiert (TanStack Virtual), Sortieren per Klick (Umschalt fügt hinzu), Breite ziehen, Spalten ausblenden — beides je Konto gemerkt (`table_views`). Tastatur:

Pfeile, Pos1/Ende, Bild auf/ab, Eingabe öffnet die Zeile. Filterleiste mit Chips, freiem WHERE, Suche und Zeilenzahl. Zeilen-Detail als Seitenblatt mit «verweist auf … öffnen» und «wird verwiesen von … 3 Zeilen zeigen». Telefon: Karten mit «alle Felder».

Zustände (O-09). Leere Tabelle, kein Treffer mit «Filter zurücksetzen», abgelehntes WHERE mit Meldung des Servers, Zeitlimit, abgerissene Verbindung und abgelaufene Sitzung beim Nachladen — je mit Ursache und nächstem Schritt.

Reiter (O-03). Jede geöffnete Tabelle oder Sicht wird ein Reiter mit ihrer letzten Adresse samt Filtern; gespeichert im Browser je Verbindung, übersteht das Neuladen. **Kürzel (O-05):** Übersicht über «?» oder das Tastatur-Symbol.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
D-01	Tests je Treiber mit Sortierung inkl. NULL, Reihenfolge ausdrücklich	`TableDataTest`: NULL zuletzt in beiden Richtungen, absteigend = Umkehrung von aufsteigend, Blättern ohne Lücken und Doppel (einfacher und zusammengesetzter Schlüssel, ohne Schlüssel), stabiler Versatz
D-02	je Treiber, mit Umlauten und Werten, die wie SQL aussehen	`x' or '1'='1`, `'; drop table people; --`, `%`, `_`, «müller»/«MÜLLER»/«jürg», und/oder, NULL, freies WHERE inkl. `1 = 1; delete from people` (nichts gelöscht)
D-03	Test je Treiber	Beschriftung des Fremdschlüssels, verweisende Zeilen gezählt, Zeile mit Verweisen in beide Richtungen
D-06	Test mit Schwelle	geschätzt über der Grenze, genau darunter, gefiltert offen, «genau zählen»; SQLite zählt mangels Schätzung immer genau
N-01	Messung mit erzeugten Daten je Treiber	`GridPerformanceTest` (`PHPDBADMIN_LEISTUNG=1`), Zahlen unten
O-03	Durchlauf	im Browser: zwei Reiter, Neuladen, beide da
O-05	Durchlauf nur mit Tastatur	Raster mit Pfeilen, Eingabe, Escape; Kürzel-Dialog über das Symbol. Die Taste «?» selbst liess sich mit dem Browser-Werkzeug nicht auslösen (es erzeugt kein `keydown`) — **von Hand offen**
O-09	leere Tabelle, SQL-Fehler, getrennte Verbindung, abgelaufene Sitzung	`ObjectControllerTest` (ungültiges WHERE → Fehlerkarte, 401/404/422/503 der JSON-Antworten), Oberfläche erklärt jede Ursache

N-01: erste Seite einer Tabelle mit einer Million Zeilen (lokal, Herd)

Fall	MySQL 8.4	MariaDB 10.11	PostgreSQL 18	SQLite
ohne Sortierung	6,5 ms	6 ms	17,1 ms	6,6 ms
`created_at` absteigend (Index)	1,5 ms	2,3 ms	2,4 ms	5,9 ms
Filter `category = 42` (ohne Index)	4,5 ms	4,5 ms	20,3 ms	35,5 ms
Suche über Textspalten	188,5 ms	138 ms	75,4 ms	156,7 ms

Zeit im Dienst inklusive Spalten, Zählung bzw. Schätzung und Beschriftungen.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
05 Kopf	«Zeile einfügen», «Exportieren», Änderungsleiste, «Rückgängig machen» fehlen	Bearbeiten TB 04, Export TB 06
05 Navigator	«Gespeichert» und «Verlauf» fehlen	SQL-Editor TB 03
07 Detail	lesend; «Duplizieren», «Zum Löschen markieren», «Zeile kopieren als» fehlen	TB 04 (D-04, E-05)
05 Raster	Spaltenkopf mit Typ statt Kurzform «decimal(10,2)» gekürzt	Breite der Spalte; der volle Typ steht in der Struktur

Tests

`TableDataTest` (52 = 13 × 4 Treiber), `ObjectControllerTest` (13), `GridPerformanceTest` (4, nur auf Wunsch). Suite: 393 Tests, 381 bestanden, 12 übersprungen (schwere Proben).

Commits

`88a95d7` Datendienst · `b2842e7` Raster und Detail · `88dec69` Reiter und Kürzel · `8af0231` Messung N-01. Die Raster-Probe der Feasibility (`/probe/raster`) ist entfernt.

Gelernt

- `rsync -R --delete` löscht auch in den Elternordnern des relativen Pfads — im Prüf-Workspace verschwanden so `database/database.sqlite`, Factories und Seeder (nur dort; Lektion im Vault).
- Larastan verlangt für rohes SQL ein Literal; statt die Prüfung zu umgehen, markiert `TrustedSql` die bewussten Stellen sichtbar.
- PostgreSQL schneidet als Text gebundene Binärdaten am Nullbyte ab — Testdaten für `bytea` als Literal setzen.

Anhang O — Timebox 03 — SQL-Editor und Befehlspalette → Alpha

Stand: 29.10.2026 · ✅ abgeschlossen — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen gebaut. Q-07 (gefährliche Anweisungen, eigentlich TB 04) ist mitgekommen, weil der Editor es braucht.

Ziel

SQL schreiben und ausführen wie in einem Desktop-Werkzeug — mit Grenzen, die eine Datenbank in Produktion schützen, und Abbrechen, das wirklich abbricht.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Zerlegen, einordnen, ausführen	Q-02, Q-03, Q-07, Q-09	✅ `StatementSplitter`, `StatementClassifier`, `QueryRunner`, Arbeitsprozess für SQLite
2 Editor, Verlauf, Palette	Q-01, Q-04, O-02	✅ Wireframes 10 und 12

Was entstanden ist

Zerleger. Eine Anweisung je Aufruf (die Verbindungen nehmen nie mehrere an, F-7). Kennt Zeichenketten mit den Escapes des Treibers (Backslash bei MySQL/MariaDB, `E'…'` bei PostgreSQL), Bezeichner (`"…"`, `` `…` ``, `[…]` bei SQLite), Kommentare (`--`, `#` bei MySQL, verschachtelte Blockkommentare bei PostgreSQL), Dollar-Blöcke, `BEGIN … END` in Routinen und Triggern (`END IF/LOOP/WHILE/REPEAT/CASE` richtig gezählt) und `DELIMITER` aus MySQL-Dumps; findet die Anweisung an der Schreibmarke (Byte-Position, der Browser rechnet UTF-16 in UTF-8 um).

Einordnen. Lesend (SELECT ohne INTO, SHOW, EXPLAIN ohne ANALYZE einer Änderung, WITH ohne ändernde Teile, lesendes PRAGMA …) oder schreibend; gefährlich sind DROP, TRUNCATE, DELETE und UPDATE ohne WHERE auf oberster Ebene und ALTER … DROP — Kommentare, Zeichenketten und Unterabfragen zählen nicht.

Ausführen. Lesendes immer im Schreibschutz-Wächter, Schreibendes nur mit Recht «schreiben» auf beschreibbaren Verbindungen und mit Protokolleintrag ohne Werte. Zeitgrenze `PHPDBADMIN_EDITOR_TIMEOUT` (30 s), Zeilen je Ergebnis `PHPDBADMIN_EDITOR_ROWS` (1'000, «mehr laden» bis 10'000). MySQL/MariaDB lesen ungepuffert und verwerfen den Rest mit `KILL QUERY`, PostgreSQL liest über einen Server-Cursor — nie liegt ein ganzes grosses Ergebnis im Speicher. **Abbrechen** ist eine zweite Anfrage: Ein Merker im Cache entscheidet über «abgebrochen» (MySQL kehrt nach `KILL QUERY` auch ohne Fehler zurück, F-2), die laufende Anweisung wird über eine eigene Verbindung beendet. **SQLite (Q-09)** läuft in einem Arbeitsprozess (`phpdbadmin:sqlite-worker`), den Zeitgrenze oder Abbruch beenden; fertige Ergebnisse davor bleiben.

Editor (Wireframe 10). CodeMirror 6 mit dem Dialekt des Treibers, Vervollständigung aus dem Schema (eine Abfrage je Treiber, als Deferred-Prop nachgeladen), Hervorhebung mit den Farben des Stylesheets, Formatieren ohne neue Abhängigkeit. ⌘↵ Anweisung an der Schreibmarke oder Auswahl, ⇧⌘↵ alles.

Ergebnisse als Reiter im virtualisierten Raster, als TSV kopieren. Gefährliches fragt nach, in Produktion mit dem Namen der Verbindung. Abfragen sind Reiter wie Tabellen, ihr Text bleibt im Browser; der Objektkopf hat den Reiter «SQL» mit einer Vorlage.

Verlauf und gespeicherte Abfragen (Q-04). `query_history` je Konto und Verbindung mit Suche (Frist 90 Tage, täglicher `model:prune`), `saved_queries` mit Namen.

Befehlspalette (O-02, Wireframe 12). ⌘K oder Suchfeld im Kopf: Objekte der offenen Verbindung, zuletzt Geöffnetes, andere Verbindungen (Suchquelle `/palette`, nur sichtbare), Aktionen («Neue Abfrage auf …», «Struktur von …», Server, Tastenkürzel); «>» zeigt nur Aktionen; Pfeile, Eingabe, Escape.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
Q-01	Durchlauf je Treiber	im Browser mit PostgreSQL: Vorlage, Formatieren, Ausführen, Vervollständigung (Endpunkt-Test: Spalten der Tabellen); Hervorhebung je Dialekt über `@codemirror/lang-sql`
Q-02	Abbrechen beendet die Abfrage auf dem Server; «abgebrochen» auch bei MySQL ohne Fehler	`QueryRunnerTest`: zweiter Prozess mit `sleep(20)` / `pg_sleep(20)` endet nach `KILL QUERY` / `pg_cancel_backend` in unter 5 s (MySQL, MariaDB, PostgreSQL); Merker entscheidet über «abgebrochen» (vier Treiber)
Q-03	lange und schreibende Anweisung auf geschützter Verbindung je Treiber	Zeitgrenze 1 s auf vier Treibern (MySQL mit grossem Kreuzprodukt, weil `SLEEP` still abbricht); Löschen auf schreibgeschützter Verbindung abgewiesen; ohne Recht nicht ausgeführt
Q-04	Test	`SqlControllerTest`: Verlauf nur des eigenen Kontos, Suche, gespeicherte Abfragen fremd 404
Q-07	Varianten mit Kommentaren, Gross-/Kleinschreibung, mehreren Anweisungen	`StatementClassifierTest` (13 Fälle Gefahr, 15 Fälle Art), `SqlControllerTest`: ohne Bestätigung 409 und nichts geändert, Produktion mit falschem Namen 409
Q-09	Abfrage über der Grenze beendet, Oberfläche meldet es	SQLite-Arbeitsprozess mit Zeitgrenze 1 s und mit Abbruch-Merker
O-02	Test der Suchquelle; Durchlauf nur mit Tastatur	`PaletteControllerTest` (nur sichtbare Verbindungen, keine Geheimnisse); im Browser ⌘K, Tippen, Auswahl

Alpha-Kriterium «Hoang nutzt phpdbadmin eine Woche zum Nachsehen»: **offen (Hoang)**.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
10 Werkzeugleiste	Auswahl des Schemas als Liste (bei mehreren)	Wireframe zeigt eine Verbindung mit einer Datenbank
10 Ergebnis	«Ergebnis exportieren» fehlt	Export TB 06
12 Palette	«Andere Verbindungen» zeigt Verbindungen, nicht ihre Tabellen	Tabellen anderer Verbindungen hiessen, jede zu öffnen — Suchquelle bewusst schlank

Tests

`StatementSplitterTest` 22, `StatementClassifierTest` 28, `QueryRunnerTest` 28 (vier Treiber), `SqlControllerTest` 8, `PaletteControllerTest` 2. Suite nach TB 03: 611 Tests, 595 bestanden, 16 übersprungen (schwere Messungen).

Commits

`efde8fe` Zerlegen, einordnen, ausführen · `d91231a` Editor, Verlauf, Palette. Die Editor-Probe der Feasibility ist entfernt.

Gelernt

- Ein eigener Hilfsname `run()` in einem PHPUnit-Test überschreibt die finale Methode der Testklasse — Fatal Error beim Laden.
- PostgreSQL: Nach einem Fehler im Cursor ist die Transaktion abgebrochen; ein `close` im `finally` wirft dann einen neuen Fehler und überdeckt den eigentlichen (Zeitlimit wurde zu «Fehler»).
- Unter PHP-FPM zeigt `PHP_BINARY` auf FPM — für einen Arbeitsprozess die CLI suchen (Einstellung `PHPDBADMIN_PHP_BINARY`).
- Der eingebaute Browser ist zwischen parallelen Helfern geteilt; Durchläufe macht nur der Hauptstrang.

Anhang P — Timebox 05 — Struktur ändern

Stand: 29.10.2026 · ✅ gebaut und getestet in drei Scheiben auf einem eigenen Zweig (Arbeitskopie), parallel zu TB 04 — Browser-Durchlauf und Zusammenführen im Hauptstrang offen.

Ziel

Tabellen anlegen, ändern, umbenennen, kopieren, leeren und löschen sowie Datenbanken bzw. Schemas anlegen und löschen — auf allen vier Treibern mit einer Vorschau, die genau das zeigt, was danach läuft, und angewendet so, dass keine Daten verloren gehen (ADR-003, N-03).

Umfang

Nr.	Anforderung (CR-001 § 4.5, § 4.10)	Prio	Ergebnis
S-02	Tabelle anlegen mit geführtem Formular (Spalten, Primärschlüssel, Indizes, Fremdschlüssel) und SQL-Vorschau	M	✅
S-03	Tabelle ändern: Spalten hinzufügen, ändern, umbenennen, löschen; Indizes und Fremdschlüssel; Kommentare — je Treiber angewendet	M	✅
S-04	Tabelle umbenennen, kopieren (Struktur, mit Daten), leeren, löschen — Leeren und Löschen nur mit Namenseingabe	M (Plan: S ↓)	✅
S-05	Datenbank bzw. Schema anlegen und löschen	S	✅ (SQLite: nur anlegen, siehe Entscheide)
N-03	Kein Datenverlust durch die Anwendung: Vorschau, Transaktion wo möglich, DDL ohne Transaktion ehrlich angezeigt, SQLite-Umbau nie mit aktiven Fremdschlüsseln	M	✅ für Strukturänderungen (Datenänderungen: TB 04)

Wireframes 9 (Tabelle ändern, MySQL und PostgreSQL, Telefon) und 6 (SQL-Vorschau als Dialog).

Was entstanden ist

Planen (`app/Services/Schema`). `SchemaPlans` baut aus geprüften Eingaben einen `SchemaPlan`: Aktion und Ziel fürs Protokoll, die Anweisungen, das Verfahren (`ApplyMode`), Hinweise, was verloren geht, und welchen Namen man zur Bestätigung eintippt. Anlegen und Ändern kommen aus Laravels `Blueprint::toSql()` — nie aus `pretend()` (F-5) —, umbenennen, kopieren, leeren, löschen und Schemas baut der Plan selbst, Namen je Segment gequotet (`SchemaName`). Jeder Name eines bestehenden Objekts muss im Katalog stehen. Vorschau und Anwenden planen über dieselbe Methode; die Vorschau über die Leseverbindung, das Anwenden noch einmal über die beschreibbare (`withWritable()`).

Ändern in einer Reihenfolge für alle vier. Neue Spalten (MySQL/MariaDB mit `AFTER`), Fremdschlüssel weg, Spalten ändern (unter altem Namen), neue Fremdschlüssel, umbenennen, Indizes weg, Spalten weg, neue Indizes, Tabellenkommentar. So baut SQLite die Tabelle höchstens einmal um, und zwar vor jedem Umbenennen.

Anwenden je Treiber (`SchemaChanges`, ADR-003).

Treiber	Verfahren	Nach einem Fehler
MySQL, MariaDB	Anweisung für Anweisung, ohne Transaktion	angehalten; die Oberfläche zeigt je Anweisung «gelaufen», «gescheitert», «nicht gelaufen»
PostgreSQL	alles in einer Transaktion	nichts angewendet
SQLite	`PRAGMA foreign_keys = OFF` ausserhalb, Umbau in der Transaktion, `PRAGMA foreign_key_check` (nur neue Verletzungen zählen), Commit, `ON` — auch nach einem Fehler	nichts angewendet

Beim SQLite-Umbau legt Laravel nur die Indizes neu an. Dazu kommen jetzt die Check-Constraints (in die neue Tabelle) und — nach sqlite.org — alle Sichten und Trigger der Datei: vorher gelöscht, danach mit ihrem gespeicherten Text neu angelegt. Ohne das scheitert das Umbenennen an jeder Sicht auf die Tabelle («error in view …: no such table»), und die Trigger der Tabelle gingen mit `DROP TABLE` verloren.

Seiten und Dialoge.

Methode	Adresse (unter `c/{connection}`)	Name
GET	`s/{schema}/new-table`	`workspace.table.create` (Seite `workspace/TableCreate`)
POST	`s/{schema}/new-table/preview`	`workspace.table.preview` (JSON)
POST	`s/{schema}/new-table`	`workspace.table.store`
GET	`s/{schema}/o/{object}/alter`	`workspace.object.alter` (Seite `workspace/TableAlter`)
POST	`s/{schema}/o/{object}/alter/preview`	`workspace.object.alter.preview` (JSON)
POST	`s/{schema}/o/{object}/alter`	`workspace.object.alter.apply`
POST	`s/{schema}/o/{object}/actions/preview`	`workspace.object.actions.preview` (JSON)
POST	`s/{schema}/o/{object}/actions`	`workspace.object.actions.apply` (umbenennen, kopieren, leeren, löschen)
GET	`schemas/sqlite-folders`	`workspace.schemas.folders` (JSON, nur SQLite)
POST	`schemas/preview`	`workspace.schemas.preview` (JSON)
POST	`schemas`	`workspace.schemas.apply`

Einstieg: «Tabelle ändern» und ein Menü «Weitere Aktionen» oben auf der Struktur-Seite, «Tabelle anlegen» und ein Menü für Datenbank bzw. Schema (SQLite: neue Datei) auf der Schema-Seite. Ohne Recht «schreiben» oder bei Schreibschutz bleiben die Knöpfe sichtbar, aber gesperrt, mit Grund daneben; die Seiten selbst antworten dann mit 403.

Rechte und Protokoll. Jede Änderung braucht die Policy-Fähigkeit `write` (Form-Request bzw. `Gate::authorize`), nie auf einer schreibgeschützten Verbindung (403 mit Klartext), eine fremde Verbindung ist 404; auf Produktion zusätzlich der Name der Verbindung (nur beim Anwenden). Leeren, Löschen und Spalten löschen verlangen den Namen der Tabelle bzw. des Schemas — geprüft in `SchemaChanges` vor der ersten Anweisung, ohne Eintrag im Protokoll. Jede angewendete Änderung ergibt genau einen Eintrag (`schema.create`, `schema.alter`, `schema.rename`, `schema.copy`, `schema.truncate`, `schema.drop`, `database.create`, `database.drop`), gelungen oder gescheitert, mit Verfahren, Zahl der gelaufenen Anweisungen und dem SQL über `SqlMask::mask()`; `AuditLog::record()` wird mit benannten Argumenten gerufen (passt zur neuen Signatur aus TB 07).

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
S-02	Vorschau (aus `toSql()`) = ausgeführtes SQL; Tabelle danach per Introspektion gleich der Eingabe	`SchemaChangesTest` je Treiber: Mitschnitt der Anweisungen = Vorschau; Spalten, NULL, Typen je Treiber, Auto-Increment, Primärschlüssel, Index, eindeutiger Index, Fremdschlüssel mit `ON DELETE CASCADE`. Endpunkt je Treiber mit Weiterleitung, Meldung und einem Eintrag (Vorgabe im Protokoll maskiert)
S-03	Tests je Treiber mit Elterntabelle und abhängigen Zeilen: Daten, Indizes, Fremdschlüssel und Kinder bleiben	`SchemaChangesTest` je Treiber: Spalte ändern, umbenennen, löschen, hinzufügen, Index anlegen an der Elterntabelle — 2 von 2 Kindern, Daten, eindeutiger Index und Fremdschlüssel der Kinder bleiben; SQLite: `foreign_key_check` leer, Fremdschlüssel wieder an; SQLite behält Checks, Trigger und Sichten. Rundlauf je Treiber: alle Typen der Liste auf ihre eigene Definition geändert — Typ, NULL und Vorgabe unverändert
N-03	Test mit erzwungenem Fehler mitten in einer Folge	je Treiber: neue Spalte, dann eindeutiger Index auf doppelte Werte — MySQL/MariaDB: 1 Anweisung bleibt und wird gemeldet, PostgreSQL/SQLite: nichts angewendet; Kinder bleiben; ein Eintrag `failed`. Über den Endpunkt (MySQL): Meldung mit «gelaufen 1, gescheitert 2»
S-04	ohne korrekten Namen keine Ausführung	je Treiber: Leeren ohne Namen und Löschen mit falscher Schreibweise abgewiesen, Zeilen bleiben, kein Eintrag; mit Namen geleert bzw. gelöscht. Dazu je Treiber: Umbenennen (Fremdschlüssel der Kinder folgen), Kopieren mit Daten (gleiche Spalten, eindeutiger Index, Zähler läuft in Original und Kopie weiter), Elterntabelle mit Kindern weder leeren noch löschen
S-05	Test je Treiber (SQLite: neue Datei im erlaubten Ordner)	MySQL, MariaDB (Verbindung ohne festen Namen) und PostgreSQL (Schema): anlegen, Löschen ohne Namen abgewiesen, mit Namen gelöscht, zwei Einträge. SQLite: neue Datei mit SQLite-Kopf im erlaubten Ordner; Ordner ausserhalb, `../` im Namen, falsche Endung, bestehende Datei abgewiesen

Querschnitt (`SchemaChangeRequestTest`, alle acht ändernden Endpunkte und beide Seiten): Recht «lesen» 403, schreibgeschützt 403 auch für den Besitzer, fremde Verbindung 404 — Datenbank und Protokoll unberührt; «schreiben» genügt; Produktion erst mit dem Namen der Verbindung, die Vorschau ohne. Namen, die Laravels Grammatik zerlegen würde (Punkt, « as », «->», «*», Anführungszeichen, Semikolon, über 63 Bytes), werden vor jeder Anweisung abgewiesen.

Abweichungen von den Wireframes

Wireframe	Abweichung	Grund
9 Typ	Typ als Liste der Originalnamen des Treibers, Länge bzw. Genauigkeit in eigenen Feldern, «eigener Typ» als Freitext	eine Liste je Treiber plus freier Typ (CR S-02); Länge getrennt prüfbar
9 Reihenfolge	keine Griffe zum Verschieben; neue Spalten kommen ans Ende (MySQL/MariaDB mit `AFTER`)	Umsortieren bestehender Spalten ist nicht in S-03
9 «+ Check»	fehlt; Checks bleiben beim SQLite-Umbau erhalten, werden aber nicht bearbeitet	nicht in S-03 (S-06 bzw. SQL-Editor)
9 Reiter «Beziehungen»	fehlt	S-07 (TB 10); den Objektkopf bearbeitet der Hauptstrang
9 Verlust je Anweisung	«Werte in ≈ N Zeilen» aus der Schätzung, ohne «davon M nicht NULL»	die Zählung bräuchte einen vollen Durchlauf grosser Tabellen

Wireframe	Abweichung	Grund
9 Telefon	voller Entwurf als Karten statt nur Zusammenfassung, darüber der Hinweis «für den Desktop ausgelegt»	gleiche Seite für alle Breiten; Bearbeiten darf auf dem Telefon eingeschränkt sein (O-07)
6	Dialog für S-04 und S-05; «Tabelle ändern» zeigt die Vorschau rechts daneben (Wireframe 9)	Wireframe 9 hat Vorrang für das Ändern

Entscheide in der Timebox

Entscheid	Begründung
Enge Namensregel (`SchemaName`): kein Punkt, keine Anführungszeichen, kein « as », «->», «*», «;», keine Steuerzeichen, höchstens 63 Bytes — auch für bestehende Namen, die ein Plan einsetzt	Laravels `wrap()` zerlegt an Punkten und « as », lässt «*» ungequotet und liest «->» als JSON-Pfad; PostgreSQL kürzt längere Bezeichner still. Was die Regel nicht erfüllt, geht im SQL-Editor
Vorgaben und `enum` -Werte über `PDO::quote` als `TrustedSql`	Laravel verdoppelt bei Vorgaben nur `'` (MySQL liest `\` als Escape) und setzt `enum` -Werte gar nicht in Anführungszeichen
Eigener Typ nur in einer engen Form (Wörter, eine Klammer mit Zahlen, `[]`); die bisherige Definition einer Spalte gilt wörtlich	freier Typ ohne Weg in fremdes SQL; bestehende Spalten bleiben änderbar
Ausdruck als Vorgabe: `CURRENT_TIMESTAMP` oder wörtlich die bisherige Vorgabe der Spalte	kein freies SQL in der Vorgabe
Bestehende Spalten: Typ aus der Liste nur, wenn Laravel ihn wieder genau so erzeugt (`ColumnTypes::specOf`), sonst wörtlich	eine Änderung von Kommentar oder NULL darf den Typ nicht umbauen
Auto-Increment und Primärschlüssel nur beim Anlegen	Umstellen eines Schlüssels ist je Treiber ein eigener Umbau, nicht in S-03
Leeren und Löschen ohne CASCADE; PostgreSQL `TRUNCATE … RESTART IDENTITY`	eine Elterntabelle mit Kindern lehnt der Server ab (SQLite: `foreign_key_check`) — nie mitgelöschte Kinder; Zähler wie bei MySQL zurückgesetzt
Kopieren je Treiber: MySQL/MariaDB `CREATE TABLE … LIKE` ; PostgreSQL `LIKE … INCLUDING ALL` , eigene Sequenzen für serial, `OVERRIDING SYSTEM VALUE` , `setval` ; SQLite der gespeicherte CREATE-Text mit neuem Namen samt Indizes	gleiche Struktur, Zähler laufen weiter; Fremdschlüssel und Trigger werden nicht mitkopiert (steht im Hinweis)
S-05 MySQL/MariaDB nur über eine Verbindung ohne festen Datenbanknamen; PostgreSQL `DROP SCHEMA … CASCADE` mit Anzahl der Tabellen im Hinweis	eine neue Datenbank erschiene sonst nirgends; Löschen eines Schemas heisst «mit allem darin» wie bei MySQL
SQLite: neue Datei nur in einem erlaubten, beschreibbaren Ordner (Kopf per `VACUUM` geschrieben); Dateien löscht phpdbadmin nicht	eine SQLite-«Datenbank» ist die Datei — Löschen gehört ins Dateisystem, nicht in eine Web-Oberfläche
Eigenes Zeitlimit `PHPDBADMIN_SCHEMA_TIMEOUT_MS` (Vorgabe 10 Minuten)	MariaDBs `max_statement_time` gilt auch für `ALTER TABLE` und hätte grosse Umbauten nach 15 s abgebrochen
Recht «schreiben» (nicht «verwalten» wie in TB 09)	Struktur ändern ist Arbeit an den Daten der Verbindung, Server-Verwaltung nicht
Ein- und Mehrzahl als eigene Texte statt `trans_choice()`	deutsche Schlüssel ohne `de.json` : `trans_choice()` fällt auf die Rückfallsprache Englisch zurück

Entscheid	Begründung
Feasibility-Probe `DdlPreviewProbeTest` entfernt	ihre Fälle (Vorschau = Ausführung, Kinder bleiben, Fehler mitten in der Folge) tragen jetzt `SchemaChangesTest`; die negativen Proben (`pretend()`, naive Transaktion) prüften Laravel und SQLite, nicht phpdbadmin — Befund bleibt in der Feasibility

Tests

126 neue Tests (davon 28 Einheitstests ohne Datenbank); die Probe mit 24 Fällen entfällt. Suite gesamt 713, davon 697 bestanden und 16 übersprungen (die schweren Proben auf Wunsch). Läufe mit `PHPDBADMIN_TEST_DATABASE=phpdbadmin_test_struktur`; Datenbanken aus S-05 heissen `phpdbadmin_s05_…` und werden im Test wieder entfernt (`finally`).

Datei	Tests	MySQL	MariaDB	PostgreSQL	SQLite
`Services/Schema/SchemaChangesTest`	17	4	4	4	5
`Http/Controllers/Workspace/Schema/TableCreateControllerTest`	15	1	1	1	12
`Http/Controllers/Workspace/Schema/TableAlterControllerTest`	7	1	—	—	6
`Http/Controllers/Workspace/Schema/TableActionControllerTest`	18	4	4	5	5
`Http/Controllers/Workspace/Schema/SchemaControllerTest`	11	1	2	2	6
`Http/Requests/Workspace/Schema/SchemaChangeRequestTest`	29	—	—	—	29
`Http/Controllers/Workspace/WorkspaceControllerTest` (neu: `canWrite`)	1	—	—	—	1
`Unit/Services/Schema/ColumnTypesTest`	28	—	—	—	—
zusammen	**126**	**11**	**11**	**12**	**64**

Prüfkette vor jedem Commit: `vp check`, `vue-tsc`, Pint, Larastan, PHPUnit — grün; `vp build` baut.

Commits

`273eaa3` Scheibe 1: planen, je Treiber anwenden, Seiten · `3de69d9` Scheibe 2: Endpunkte je Treiber getestet, Rechte, Bestätigung · `d6f6d40` Zielspalte und Fehler der Vorschau · Doku (dieser Commit).

Offen

- **Browser-Durchlauf** bei 1440, 768 und 375 px, hell und dunkel, gegen Wireframe 9 und 6 — Hauptstrang (in dieser Arbeitskopie ohne Browser gebaut).
- Spalten umsortieren, Primärschlüssel ändern, Checks bearbeiten, Index-Arten (Volltext, räumlich); zusammengesetzte Fremdschlüssel kann der Server, die Oberfläche bietet je Fremdschlüssel eine Spalte.

- SQLite-Tabellen mit Inline-`UNIQUE` (`sqlite_autoindex_…`): Laravels Umbau will den internen Index neu anlegen (vertauschte Argumente in `str_starts_with`) und scheitert — zurückgerollt, nichts verloren, aber nicht änderbar. Nicht getestet.
- `MaintenanceController` (TB 09) nutzt `trans_choice()` und zeigt auf Deutsch englische Mehrzahl — besser an einer Stelle lösen (Basalto: `SourceLocaleTranslator`, Lektion «Deutsche Mehrzahl fällt auf Englisch zurück» im Vault).
- Laravels `compileColumns` für MySQL setzt den Tabellennamen über `quoteString()` ohne Escape ein — die Strukturansicht (S-01) scheitert an Tabellennamen mit `'`. Vorbestehend, nicht in TB 05 behoben.

Gelernt

- **Rundlauf je Treiber:** Eine Spalte auf ihre eigene Definition ändern und Typ, NULL und Vorgabe vergleichen fand sofort eine falsche Zuordnung (`tinyint(1)` wurde zu `tinyint`, weil die Ganzzahl-Regel vor dem Wahrheitswert griff). Jede Rückübersetzung aus der Introspektion braucht so einen Test.
- **SQLite prüft beim `RENAME TABLE` die ganze Datei:** Eine Sicht auf die umgebaute Tabelle liess den Umbau scheitern. Sichten und Trigger vor dem Umbau löschen und danach neu anlegen (sqlite.org, Abschnitt 7).
- **Laravels Grammatik ist kein Escaping:** `quoteString()` für `enum`-Werte verdoppelt nichts, Vorgaben nur `'` — Literale über `PDO::quote`, Bezeichner nur nach enger Regel.
- `trans_choice()` mit deutschen Schlüsseln ist eine bekannte Falle (Lektion im Vault) — und wurde trotzdem wieder gebaut, weil der Lektionen-Index nicht zu Beginn gelesen war; ein Endpunkt-Test mit der deutschen Meldung hat es gefunden.

Anhang Q — Timebox 09 — Server-Verwaltung

Stand: 29.10.2026 · ✅ gebaut und getestet in drei Scheiben auf einem eigenen Zweig (Arbeitskopie), vorgezogen vor v1.0 — Browser-Durchlauf und Zusammenführen im Hauptstrang offen.

Ziel

Den Server hinter einer Verbindung ansehen und geführt verwalten — Übersicht, Prozesse, Variablen, Konten und Rechte, Wartung —, mit derselben Bedienung für MySQL, MariaDB, PostgreSQL und SQLite und ohne dass etwas ungesehen passiert.

Umfang

Nr.	Anforderung (CR-001 § 4.8)	Prio	Ergebnis
W-01	Server-Übersicht: Version, Laufzeit, Grösse je Datenbank, Zeichensätze	S	✅
W-02	Prozesse und Sitzungen anzeigen und beenden	S	✅
W-03	Variablen und Status lesen	C	✅ (Durchlauf offen)
W-04	Benutzer, Rollen und Rechte geführt verwalten mit Vorschau	S	✅
W-05	Wartung: ANALYZE, VACUUM, OPTIMIZE, Integritätsprüfung	C	✅ (Durchlauf offen)

Kein eigenes Wireframe (Timebox-Plan: «in TB 09 entworfen»); gestaltet im Stil der Wireframes 05 (Arbeitsbildschirm), 06 (SQL-Vorschau) und 08 (Struktur).

Was entstanden ist

Dialekt (ADR-001). Der Vertrag kennt jetzt `serverOverview`, `sessionId`, `processes`, `endProcessStatement` / `endProcess`, `variables`, `statusValues`, `accounts`, `currentAccount`, `privilegeChoices`, `createAccountStatements`, `dropAccountStatements`, `privilegeStatements`, `maintenanceOperations`, `maintenanceForTables`, `maintenanceStatements` und `runMaintenance` — umgesetzt in allen vier Dialekten; SQLite gibt `null` bzw. leere Listen, wo es Prozesse und Konten nicht gibt. Was ändert, plant der Dialekt nur als `PlannedStatement` (echter Text und Text für Vorschau und Protokoll, Passwort maskiert).

Ändernde Aktionen. `TargetConnections::withWritable()` öffnet für genau eine Aktion eine Verbindung ohne Schreibschutz — nie für eine schreibgeschützte (`ReadOnlyConnection`), getrennt von den Leseverbindungen und danach sofort freigegeben. `ServerPlans` plant aus geprüften Eingaben; jeder Name eines Ziels muss im Katalog stehen. `ServerChanges` zeigt die Vorschau über die Leseverbindung und führt über die beschreibbare aus, Anweisung für Anweisung, ohne Abfrage-Ereignisse (dort stünde das Passwort); jede Aktion ergibt genau einen Eintrag im Protokoll — `server.kill`, `server.user.create`, `server.user.drop`, `server.grant`, `server.revoke`, `server.maintenance` —, gelungen oder gescheitert, mit SQL ohne Geheimnisse. `AuditLog::record()` nimmt dafür das SQL auf.

Seiten. Unter `c/{connection}/server` fünf Inertia-Seiten im Arbeitsbildschirm (`workspace/server/Overview`, `Processes`, `Variables`, `Accounts`, `Maintenance`), Einstieg «Server» im Menü der Verbindung:

Methode	Adresse	Name
GET	`server`	`workspace.server`
GET	`server/processes`	`workspace.server.processes`
POST	`server/processes/{process}/end`	`workspace.server.processes.end`
GET	`server/variables`	`workspace.server.variables`
GET	`server/accounts`	`workspace.server.accounts`
POST	`server/accounts/preview`	`workspace.server.accounts.preview` (JSON)
POST	`server/accounts`	`workspace.server.accounts.apply`
GET	`server/maintenance`	`workspace.server.maintenance`
POST	`server/maintenance/preview`	`workspace.server.maintenance.preview` (JSON)
POST	`server/maintenance`	`workspace.server.maintenance.run`

Abnahmekriterien

Anforderung	Kriterium	Nachweis
W-01	Test je Treiber	Vertrag: Version, Laufzeit (SQLite keine), Zeichensatz, Grösse der Test-Datenbank > 0, PostgreSQL zusätzlich je Schema, SQLite Seitengrösse, Journal und Datei — vier Treiber; Seite je Server-Treiber und SQLite
W-02	beendete Sitzung ist weg	Vertrag je Server-Treiber: zweite Sitzung öffnen, Kennung über `connection_id()` bzw. `pg_backend_pid()`, beenden — danach nicht mehr in der Liste und tot (eigene PDO-Verbindung scheitert), ein zweites Beenden meldet «gibt es nicht»; Abbrechen stoppt `sleep` in einer asynchronen Sitzung (mysqli, pgsql), die Sitzung bleibt. Endpunkt je Server-Treiber: Weiterleitung, Meldung, genau ein Eintrag `server.kill`. Die eigene Sitzung ist markiert und abgewiesen
W-03	Durchlauf	Vertrag vier Treiber, Seite für SQLite und PostgreSQL; Browser-Durchlauf offen
W-04	Tests je Treiber (ohne SQLite)	Vertrag je Server-Treiber: Konto mit einem Passwort, das wie SQL aussieht, anlegen, mit diesem Passwort anmelden, Recht auf eine Tabelle erteilen, mit `SHOW GRANTS` bzw. `has_table_privilege` und mit einem echten Zugriff prüfen, entziehen, löschen; Rechte auf Server, Datenbank und (PostgreSQL) Schema erscheinen in der Liste und verschwinden wieder. Endpunkt je Server-Treiber: Vorschau maskiert und ändert nichts, Lebenszyklus mit vier Einträgen im Protokoll ohne Passwort
W-05	Durchlauf	Vertrag je Treiber und Art (11 Fälle) für eine Tabelle und das ganze Schema ohne Fehler; SQLite erkennt einen beschädigten Index; Endpunkt mit Ergebnis je Tabelle, VACUUM auf SQLite trotz ATTACH-Wächter; Browser-Durchlauf offen

Querschnitt: ohne «verwalten» 403, schreibgeschützt 403 (auch für den Besitzer), fremde Verbindung 404, Produktion erst mit dem eingetippten Namen — für alle fünf ändernden Endpunkte, ohne Eintrag im Protokoll. Eingaben, die wie SQL aussehen (Kontoname, Host, Recht, Datenbank, Tabelle «*», Ebene, Prozesskennung), werden vor jeder Anweisung abgewiesen.

Gestaltungsentscheide (statt Wireframe-Abweichungen)

Bereich	Entscheid	Grund
Einstieg	Eintrag «Server» im Menü der Verbindung; Navigator unverändert, zeigt die Schemas ohne Auswahl	Navigator und Layout bearbeitet parallel jemand anderes

Bereich	Entscheid	Grund
Bereiche	Reiter wie in der Strukturansicht (Wireframe 08), brechen um statt zu scrollen	gleiche Bedienung wie im übrigen Arbeitsbildschirm
Bestätigung	Dialog im Stil der SQL-Vorschau (Wireframe 06): nummerierte Anweisungen, Hinweise je Treiber, auf Produktion Name der Verbindung, Fehler des Servers im Dialog	eine Bestätigung für Prozesse, Konten und Wartung
Konten	erst ein Formular (anlegen bzw. Ebene, Ziel, Rechte), dann die Vorschau; Löschen geht direkt zur Vorschau	nichts läuft ohne Vorschau, auch nicht «nur» ein REVOKE
Ebenen	MySQL/MariaDB: Server, Datenbank, Tabelle; PostgreSQL: Server (vordefinierte Rollen wie `pg_monitor`), Datenbank, Schema (USAGE/CREATE und Rechte auf allen Tabellen darin), Tabelle	PostgreSQL kennt keine Rechte auf «.»
Schmale Fenster	Prozesse, Konten und Variablen als Karten bzw. umbrechende Zeilen, keine breiten Tabellen	375 px lesbar (O-07)
Gesperrt	Knöpfe bleiben sichtbar, aber gesperrt, mit Grund (schreibgeschützt bzw. ohne «verwalten»)	man sieht, was ginge
SQLite	gleiche Reiter; Prozesse und Konten mit klarer Erklärung, Variablen = lesbare PRAGMAs, Status = Optionen der Bibliothek	gleiche Bildschirme für alle vier (O-04)

Entscheide in der Timebox

Entscheid	Begründung
Bezeichner je Segment selbst quoten (Anführungszeichen verdoppelt) statt `Grammar::wrap()`	`wrap()` trennt an Punkten und « as » und lässt «» *ungequotet — eine Tabelle* «» würde zu «alle Tabellen»
Zeichenketten (Passwort, Host, Kontoname bei MySQL) über `PDO::quote`, Rechte nur aus der festen Liste je Ebene	DDL wie `CREATE USER` nimmt keine Platzhalter
MySQL/MariaDB: `_` und `%` im Datenbanknamen eines GRANT maskiert	auf Datenbank-Ebene sind sie Platzhalter — «my_db» träfe auch «myXdb»
Vorschau als JSON (`useHttp`), Ausführen als Inertia-POST mit Weiterleitung und Meldung	dieselbe Planung für beide; die Seite lädt nach einer Änderung neu
Ein Eintrag im Protokoll auch für gescheiterte Versuche (`outcome: failed`, Meldung maskiert)	wer was versucht hat, gehört ins Protokoll
Meldungen des Servers aus `getPrevious()` der `QueryException`, Geheimnisse maskiert	Laravel hängt die Anweisung samt Werten an die Meldung
PostgreSQL meldet sich als `application_name = phpdbadmin`	eigene Sitzungen in `pg_stat_activity` erkennbar
PostgreSQL: `DROP ROLE` ohne `DROP OWNED`	`DROP OWNED` löscht Objekte; lieber ablehnen lassen und den Hinweis zeigen
Wartung mit eigenem Zeitlimit (`PHPDBADMIN_MAINTENANCE_TIMEOUT_MS`, Vorgabe 10 Minuten)	OPTIMIZE und VACUUM grosser Tabellen brauchen länger als eine Abfrage
SQLite-VACUUM: `ATTACH ''` im Authorizer erlaubt (Zeile aus dem Hauptstrang)	VACUUM hängt intern eine leere Datenbank an

Tests

126 neue Tests; Suite gesamt 449, davon 437 bestanden und 12 übersprungen (die schweren Proben auf Wunsch). Testläufe mit `PHPDBADMIN_TEST_DATABASE=phpdbadmin_test_server`; Konten heissen `phpdbadmin_w04_…` und werden im Test wieder entfernt (`finally`).

Datei	Tests	MySQL	MariaDB	PostgreSQL	SQLite
`Dialects/DialectServerContractTest`	46	12	15	11	8
`Http/Controllers/Workspace/Server/*` (fünf Dateien)	53	5	25	6	17
`Http/Requests/Workspace/Server/ServerChangeRequestTest`	18	—	—	—	18
`Services/Server/ServerPlansTest`	7	3	2	2	—
`Connections/TargetConnectionsTest` (neu: beschreibbare Verbindung)	2	—	—	—	2
zusammen	**126**	**20**	**42**	**19**	**45**

Prüfkette vor jedem Commit: `vp check`, `vue-tsc`, Pint, Larastan, PHPUnit — grün.

Commits

`5d54279` Scheibe 1: Dialekt für die Server-Verwaltung · `2272649` Scheibe 2: Grundlage für ändernde Aktionen · `87b5a20` Scheibe 3: Seiten und Endpunkte · Doku (dieser Commit).

Offen

- **Browser-Durchlauf** bei 1440, 768 und 375 px, hell und dunkel, gegen einen Herd-Workspace — Hauptstrang.
- **PostgreSQL schickt das Passwort im Klartext** (`CREATE ROLE … PASSWORD '…'`); mit `log_statement = ddl` steht es im Server-Log. Abhilfe wäre ein SCRAM-Verifier, im Client berechnet — PHP fehlt dafür SASLprep.
- **Rollen in MariaDB** werden gelistet, aber nicht gelöscht (`DROP ROLE`); MySQL-Rollen erscheinen als gesperrte Konten.
- **Anonyme Konten in MariaDB** (Herd: `''@'localhost'`) gehen `'app'@'%'` vor — ein neues Konto meldet sich lokal erst mit Host `localhost` an. Ein Hinweis in der Oberfläche fehlt.
- Integritätsprüfung für PostgreSQL (`amcheck`) nicht im Umfang; die Prozessliste aktualisiert sich nur auf Knopfdruck.
- Grosse Wartung kann an der Laufzeitgrenze von PHP scheitern, bevor das Zeitlimit des Servers greift.

Gelernt

- `Grammar::wrap()` ist kein Quoting für beliebige Namen (Punkte, « as », «*») — für Bezeichner aus Eingaben eigenes Quoting je Segment (Lektion für den Vault).
- MariaDB hängt an `SHOW GRANTS` den Passwort-Hash (`IDENTIFIED BY PASSWORD '*…'`) — vor der Anzeige entfernen.
- Inertia-Tests verlangen die Seitendatei: Endpunkte und Seiten gehören in dieselbe Scheibe.

- Pint (`php_unit_method_casing`) benennt private Hilfsmethoden, die mit «test» beginnen, in Testklassen in `test_…` um — Hilfsmethoden anders benennen.